IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

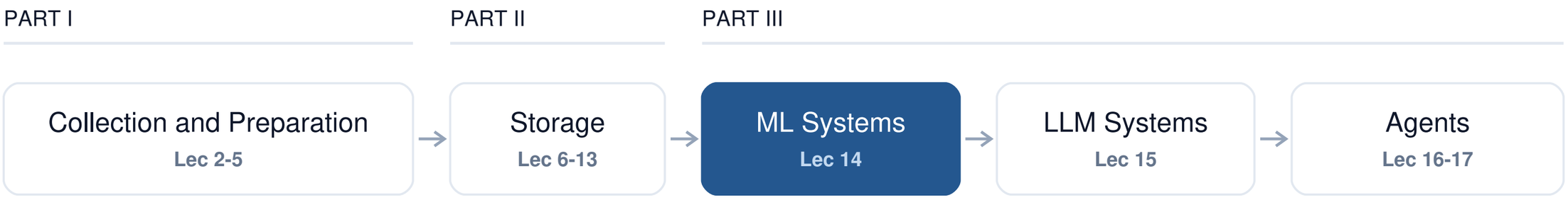
14.

ML Systems

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
Storage
Lec 6-13
ML Systems
Lec 14
LLM Systems
Lec 15
Agents
Lec 16-17

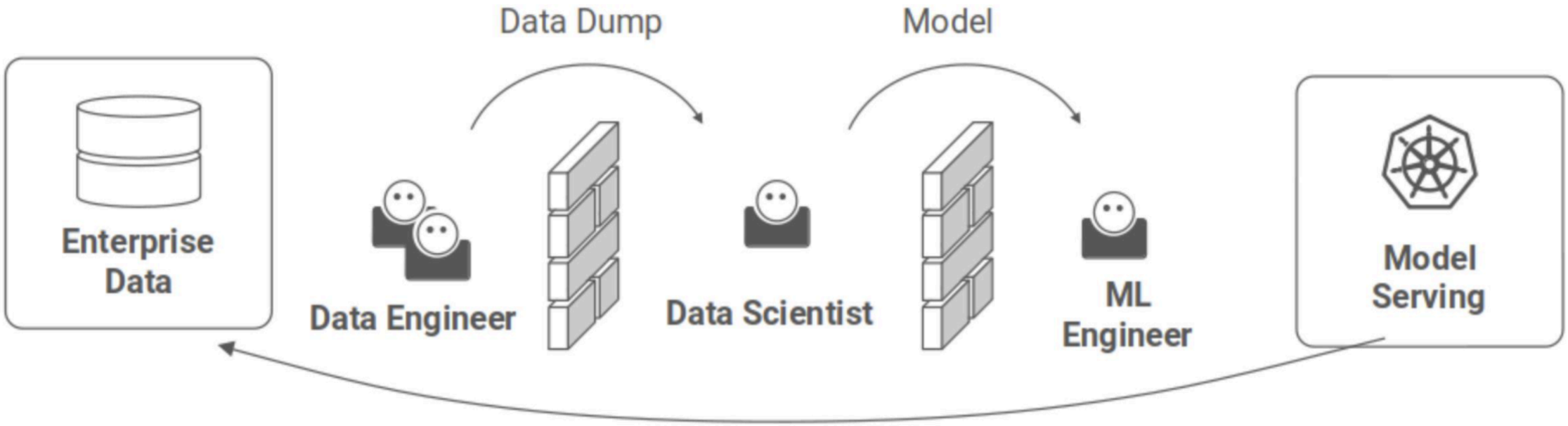
Outline: ML Systems

01	**ML Pipelines and MLOps**	from data preparation to deployed predictions
02	**Make Everything Reproducible**	recover data, features, code and models
03	**Test Every Step**	test features, models and the running system
04	**Log Everything**	observe predictions and diagnose drift

01 · ML Pipelines and MLOps

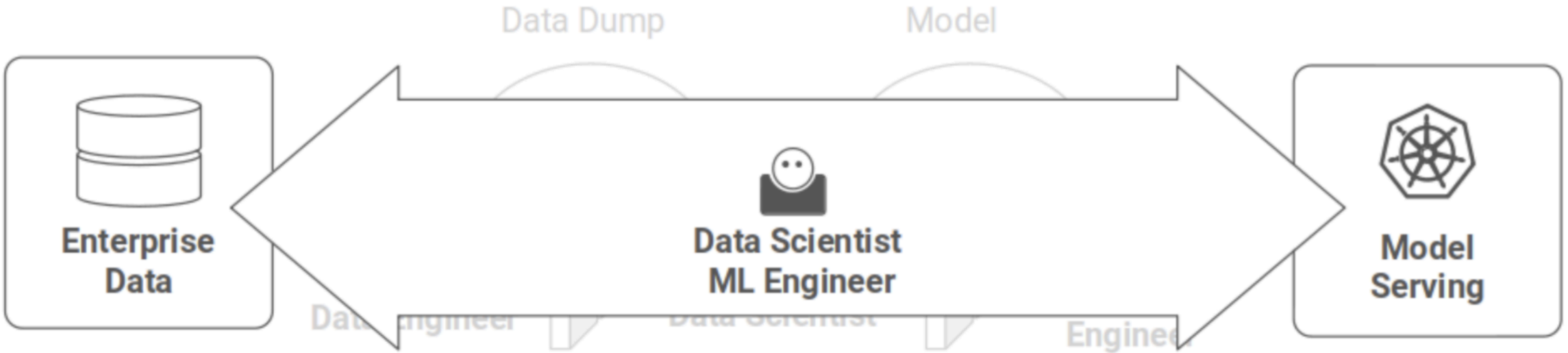
01	**ML Pipelines and MLOps**	from data preparation to deployed predictions
02	**Make Everything Reproducible**	recover data, features, code and models
03	**Test Every Step**	test features, models and the running system
04	**Log Everything**	observe predictions and diagnose drift

Data Scientist and ML Engineer



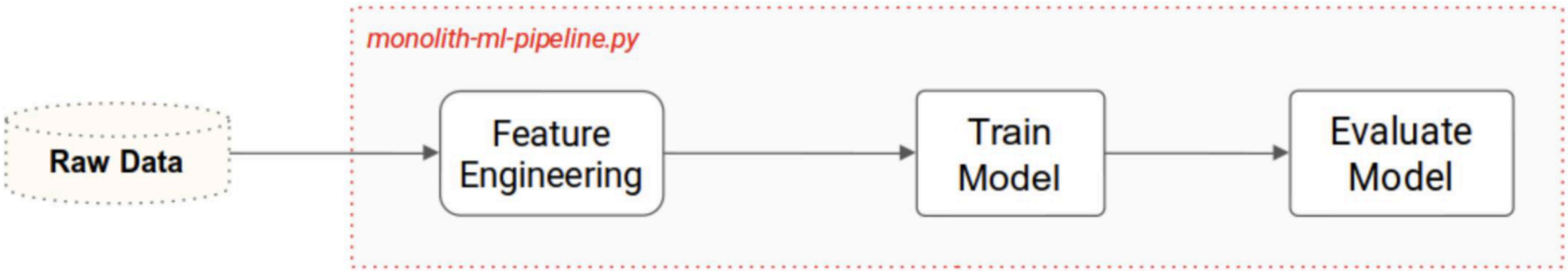
The original workflow separates data preparation, model development and serving

An End-to-End ML Workflow



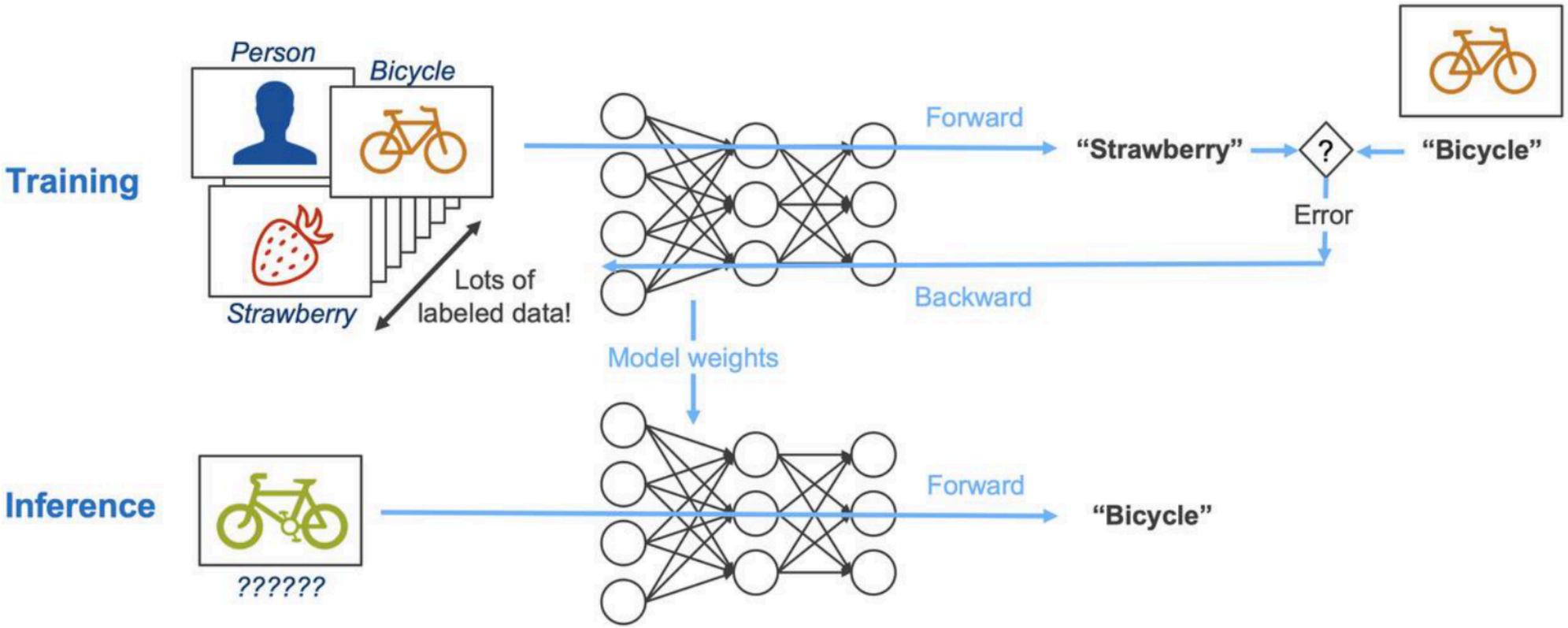
The same workflow becomes one end-to-end responsibility

ML Pipeline



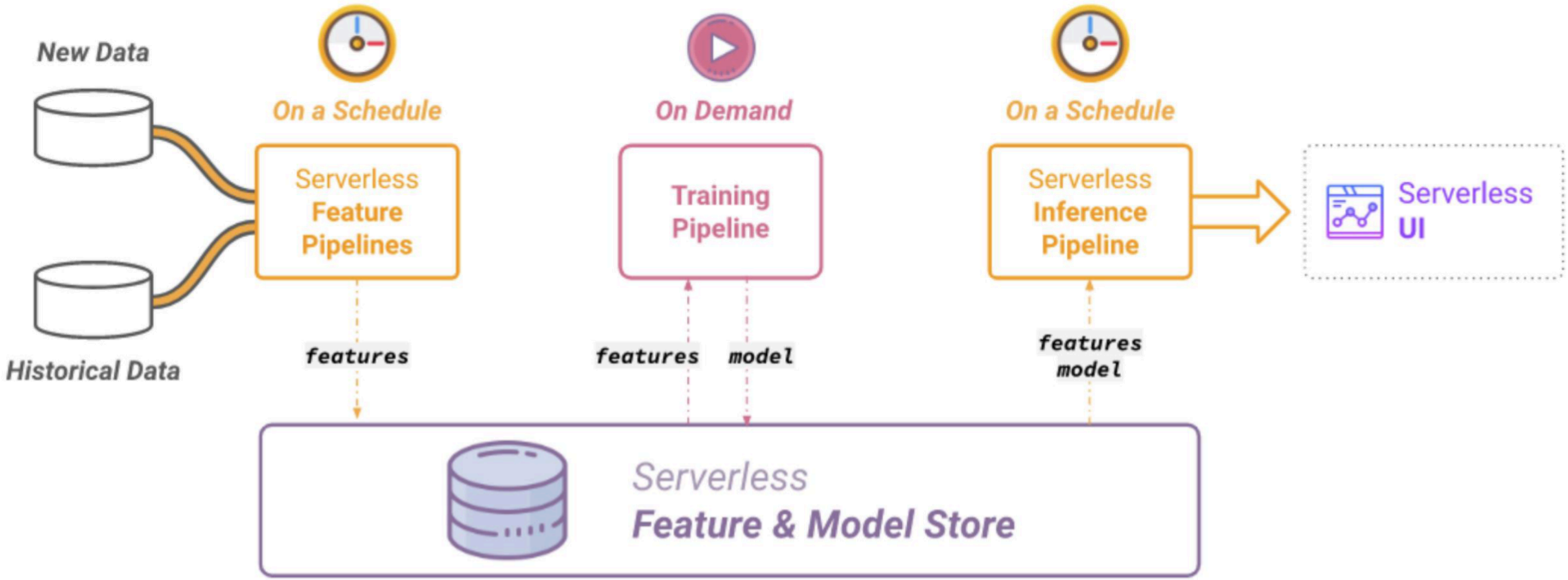
Raw data passes through feature engineering, training and evaluation

Training and Inference



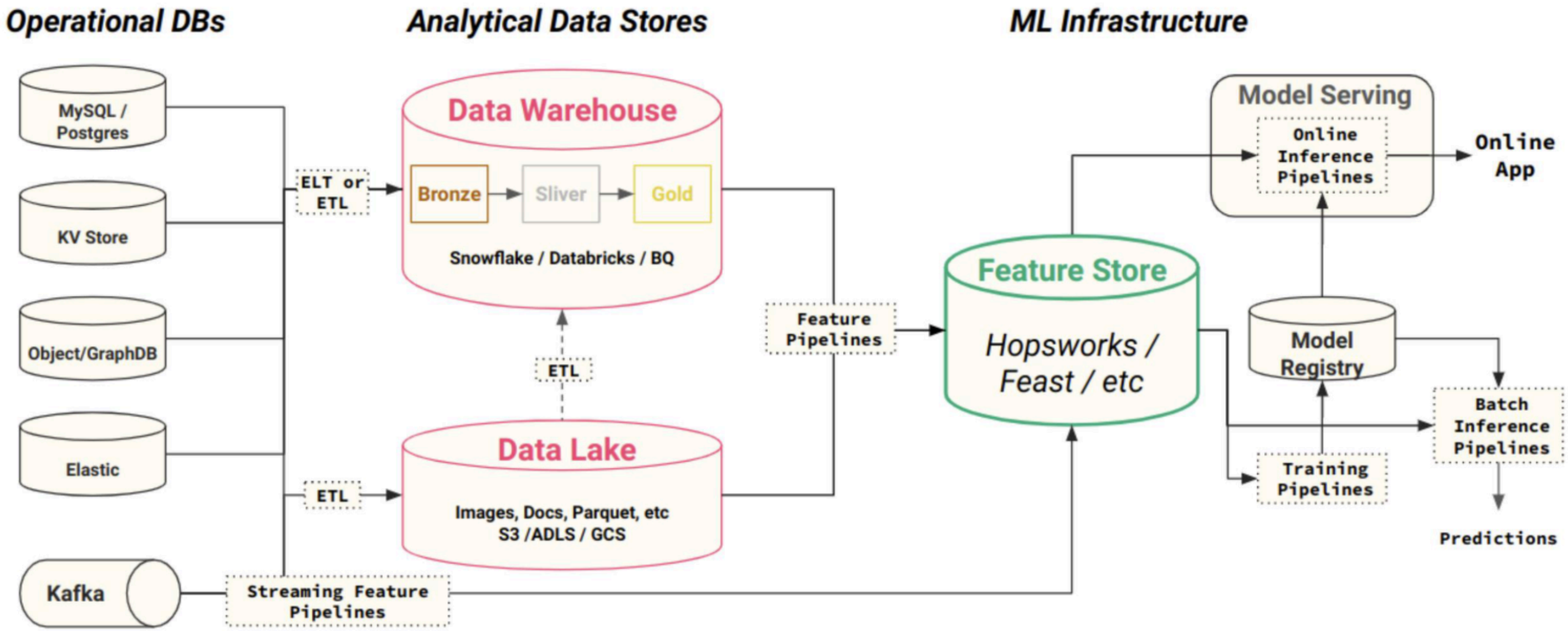
Training changes weights; inference applies the learned weights to new inputs

ML Pipelines at Scale



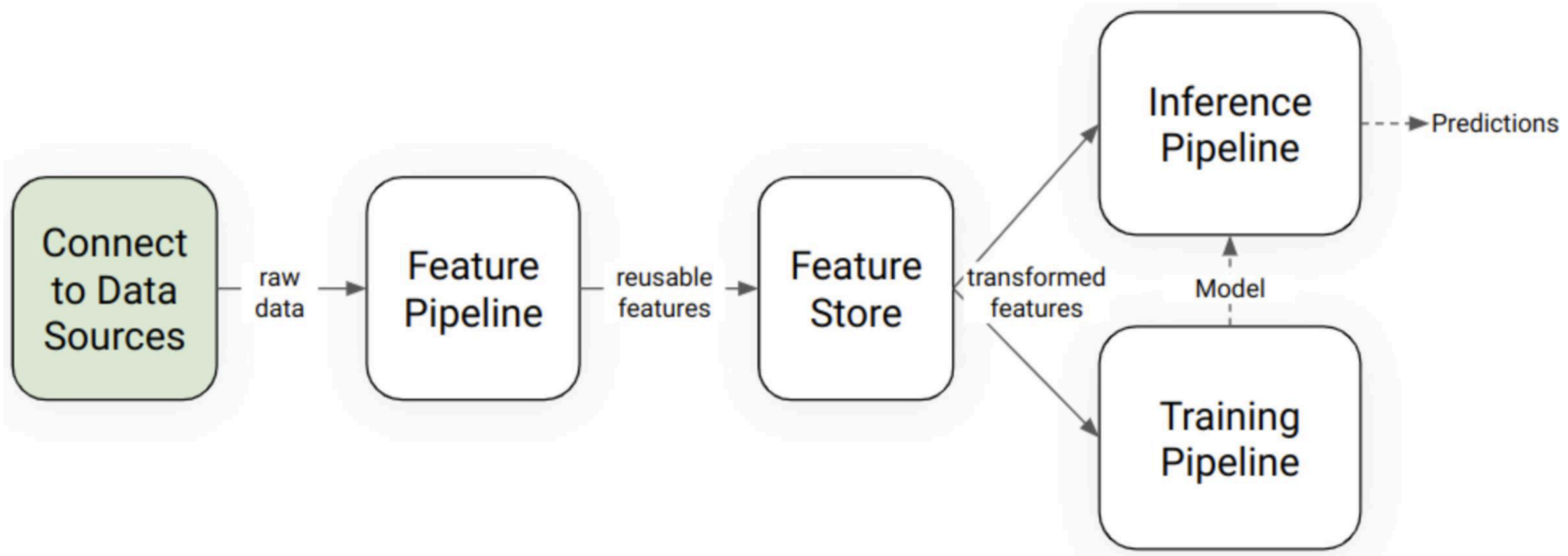
Feature, training and inference pipelines can run on different schedules

Data Stores and ML Infrastructure



Operational data, analytical storage and ML infrastructure serve different roles

ML Pipeline Stages: Data Sources



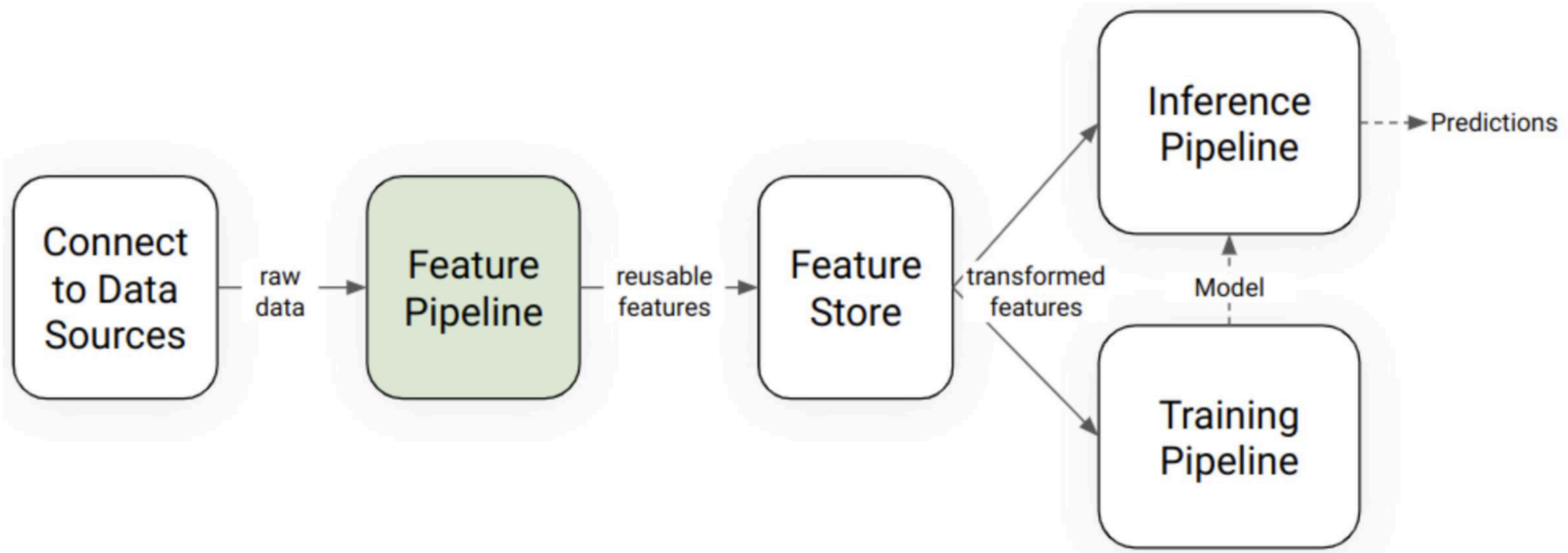
The first stage connects the pipeline to its source records

Recap: Data Collection

Type of Data	File Formats	Example Systems
Tabular data	.csv, .parquet, .tfrecord, .avro	Snowflake, Databricks, BQ, Redshift, S3, ADLS, GCS
Unstructured data	images, sound, video	S3, GCS, ADLS, HDFS
Free-text search	application/service logs	Elasticsearch, Solr
Documents	JSON	MongoDB
Graph data	Social networks	Neo4J
Time-series data	Performance metrics	InfluxDB, Prometheus
Queued data	Avro	Kafka, Kinesis
REST APIs	REST API with API key	Saas Platform
Web scraped	N/A	Websites publishing data

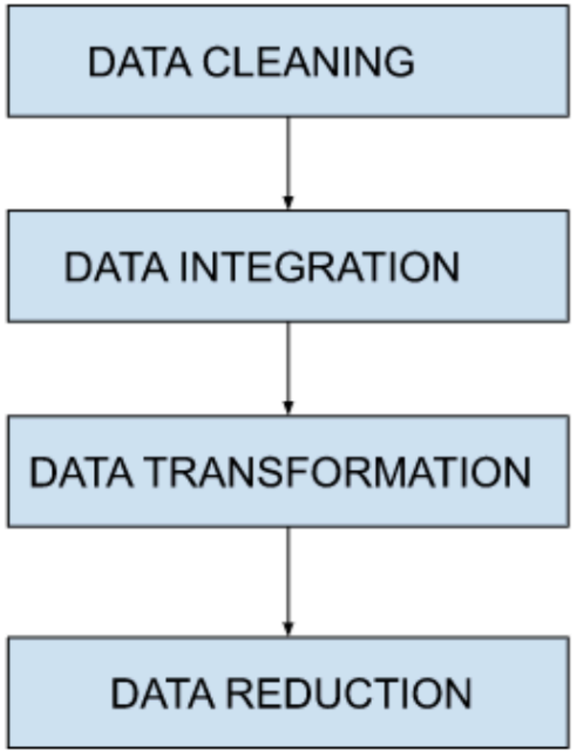
Different data types require different collection and storage interfaces

ML Pipeline Stages: Features



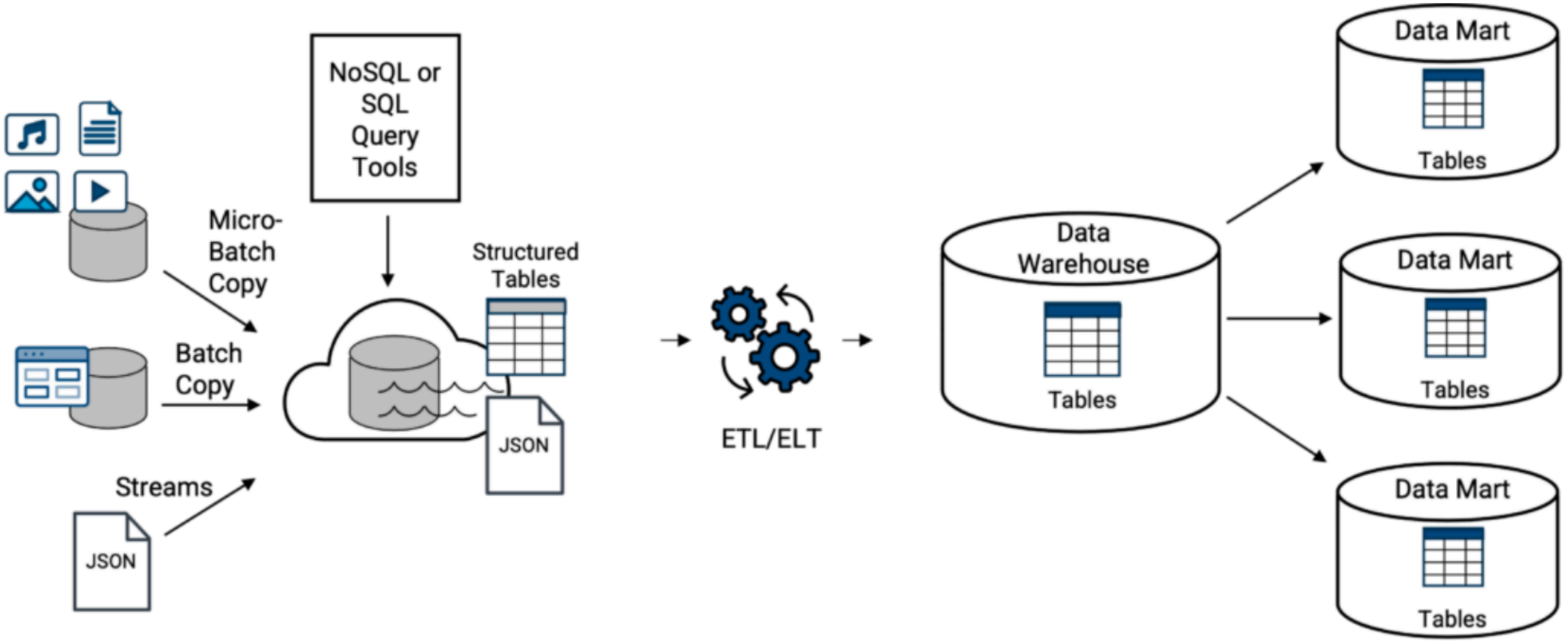
The feature pipeline turns source records into reusable model inputs

Recap: Data Preprocessing



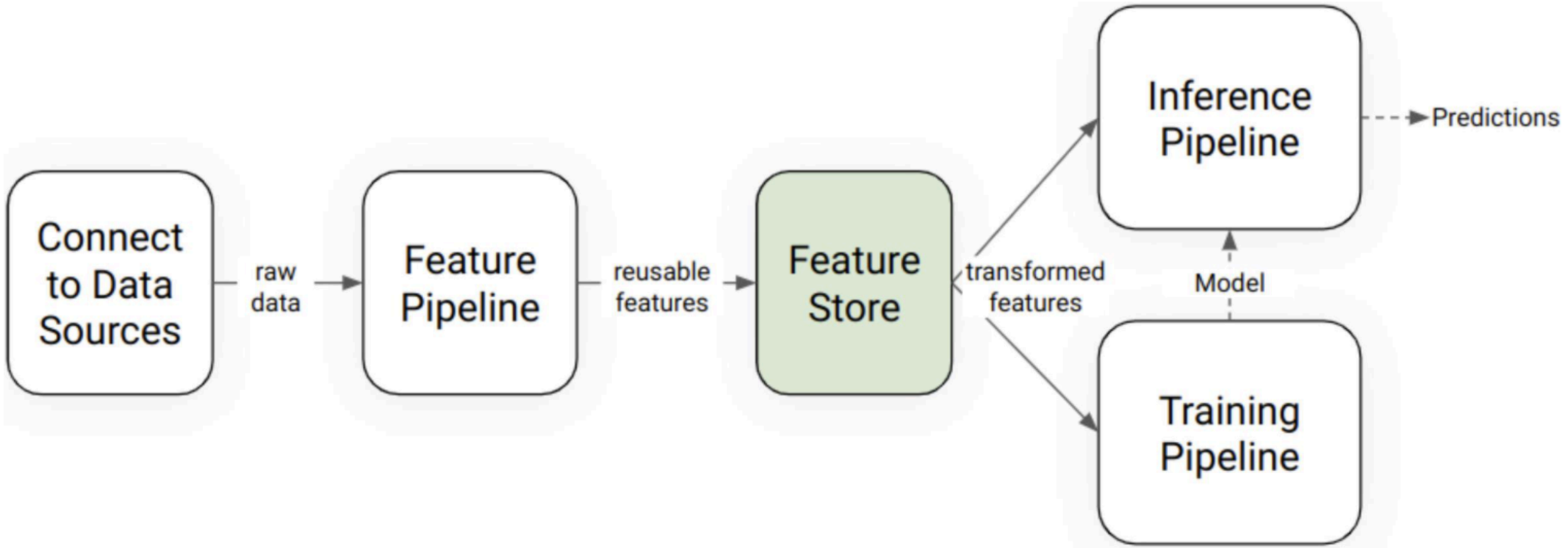
Cleaning, integration, transformation and reduction prepare usable inputs

Recap: Unstructured to Structured Data



Source records become tables before downstream analytical and ML use

ML Pipeline Stages: Feature Store



Reusable features connect feature preparation to training and inference

Recap: **Fraud Detection Data**

Data for a prediction service to identify if a credit card transaction is **suspected of fraud or not.**

transactions table

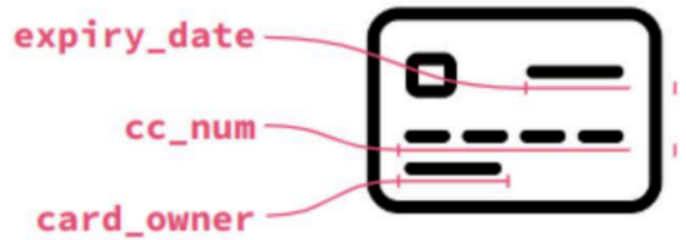


user_profile table

Name
Sex
Date of Birth
City



credit_card_details table



Fraud prediction combines transactions, profiles and credit-card details

Recap: Source Tables and Update Rates



Updated
once/hour

tid	cc_num	datetime	category	amount	long/lat	fraud
1	11111	2022-09-01 11:24	food	45.33	53N,6W	No
2	11111	2022-09-02 09:17	clothing	183.12	52N,6W	No
3	11111	2022-09-04 19:33	entertain	63.33	51N,7W	yes
..	…	…	…	…	…	…



Updated
once/week

credit_cards

cc_num	provider	expires
11111	visa	24/05
…	…	…



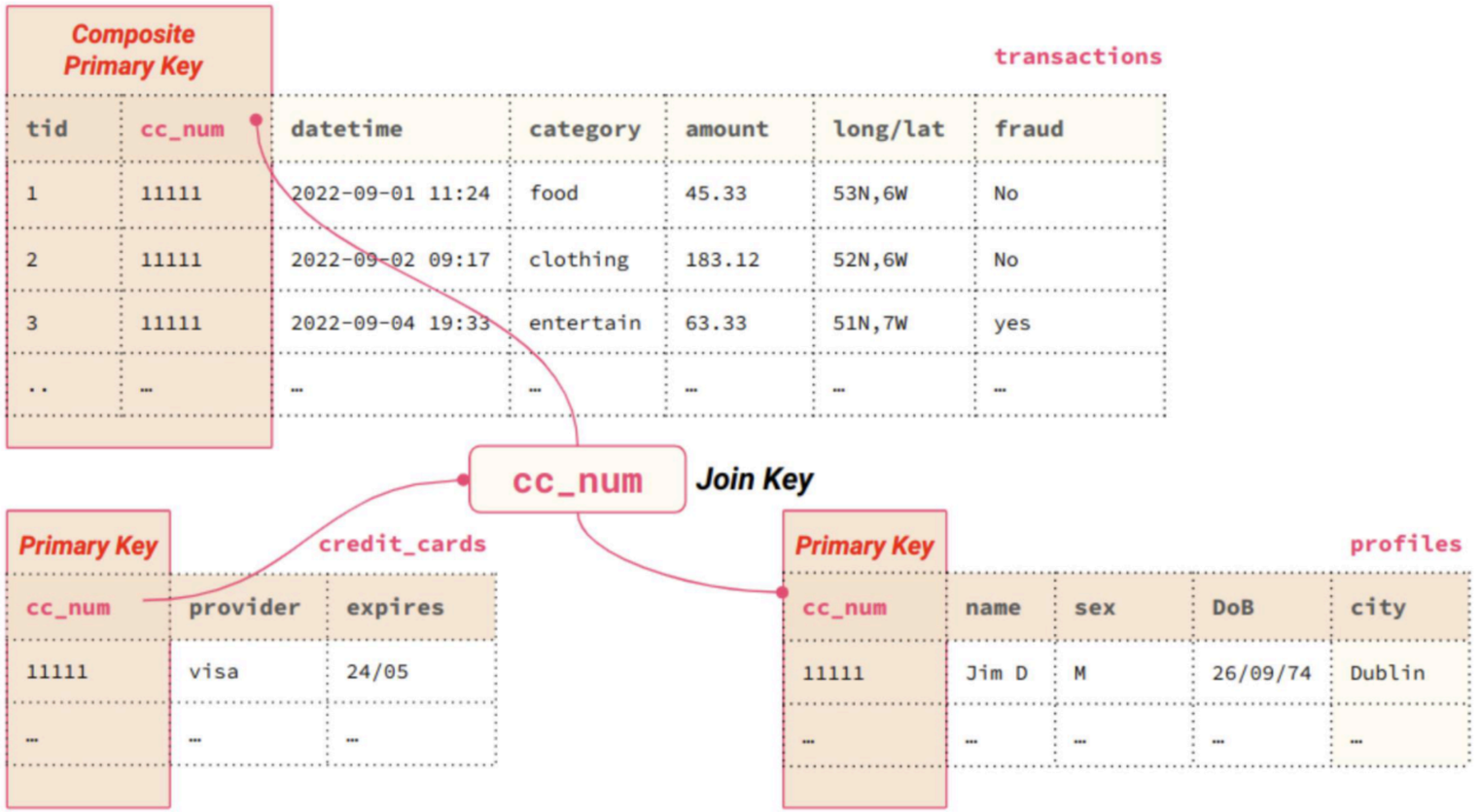
Updated
once/day

profiles

cc_num	name	sex	DoB	city
11111	Jim D	M	26/09/74	Dublin
…	…	…	…	…

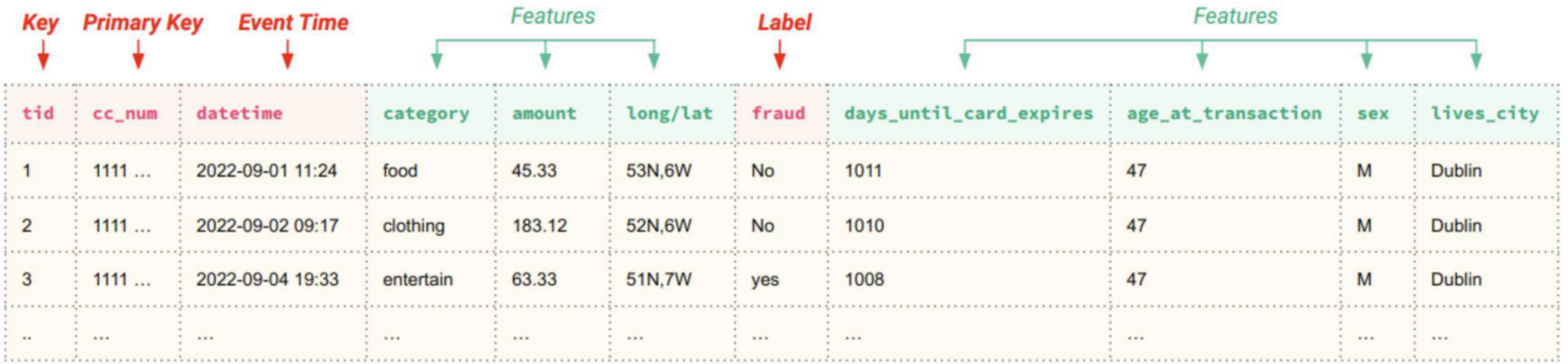
Transaction, card and profile tables are refreshed at different rates

Recap: Keys and Joins



Primary keys identify rows and join keys connect the three source tables

Recap: One Big Table



tid	cc_num	datetime	category	amount	long/lat	fraud	days_until_card_expires	age_at_transaction	sex	lives_city
1	1111 ...	2022-09-01 11:24	food	45.33	53N,6W	No	1011	47	M	Dublin
2	1111 ...	2022-09-02 09:17	clothing	183.12	52N,6W	No	1010	47	M	Dublin
3	1111 ...	2022-09-04 19:33	entertain	63.33	51N,7W	yes	1008	47	M	Dublin
..	...	...	...	...	...	...	...	...	...	...

One prediction row combines keys, event time, features and a label

Recap: Features with Different Cadences

Our main Feature group is `transactions`. The columns for this one big table (OBT) have different data sources, that are updated at different cadences. It is inefficient to update all columns in every update, but less JOINs will be required for training data.

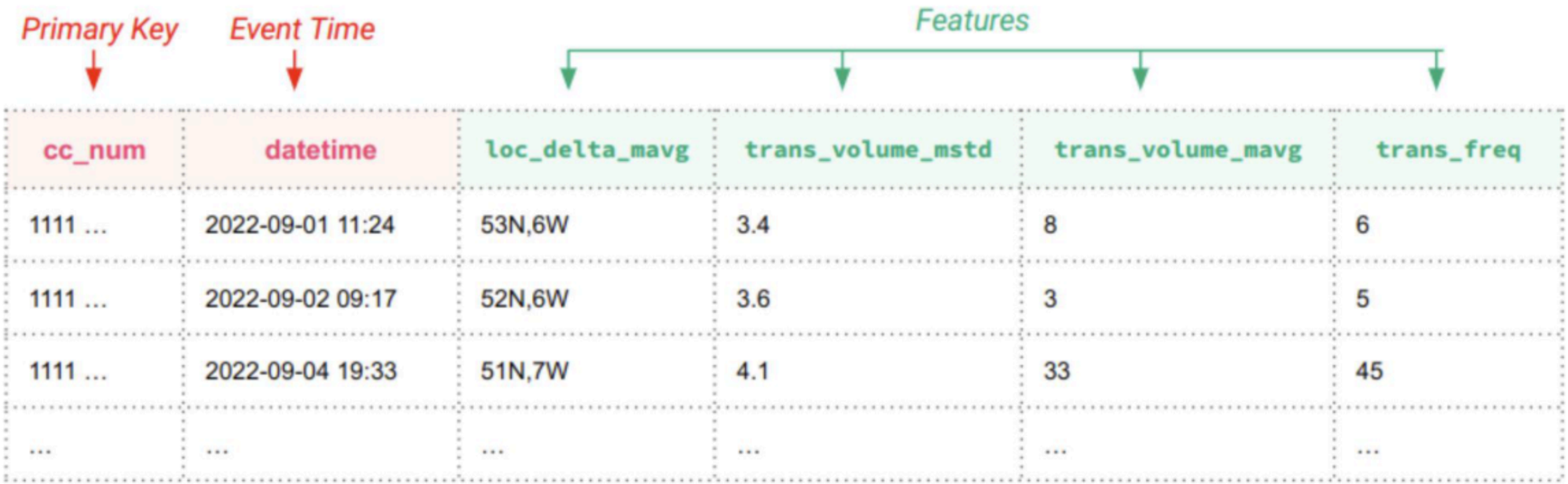
Updated Frequently | *Updated Less Frequently*

Source: cc_transactions							Source: credit_card	Source: profile		
tid	cc_num	datetime	category	amount	long/lat	fraud	days_until_card_expires	age_at_transaction	sex	lives_city
1	1111 ...	2022-09-01 11:24	food	45.33	53N,6W	No	1011	47	M	Dublin
2	1111 ...	2022-09-02 09:17	clothing	183.12	52N,6W	No	1010	47	M	Dublin
3	1111 ...	2022-09-04 19:33	entertain	63.33	51N,7W	yes	1008	47	M	Dublin
..	...	...	...	...	...	...	...	...	...	...

Feature columns from different sources need not be recomputed at the same rate

Recap: Four-Hour Transaction Features

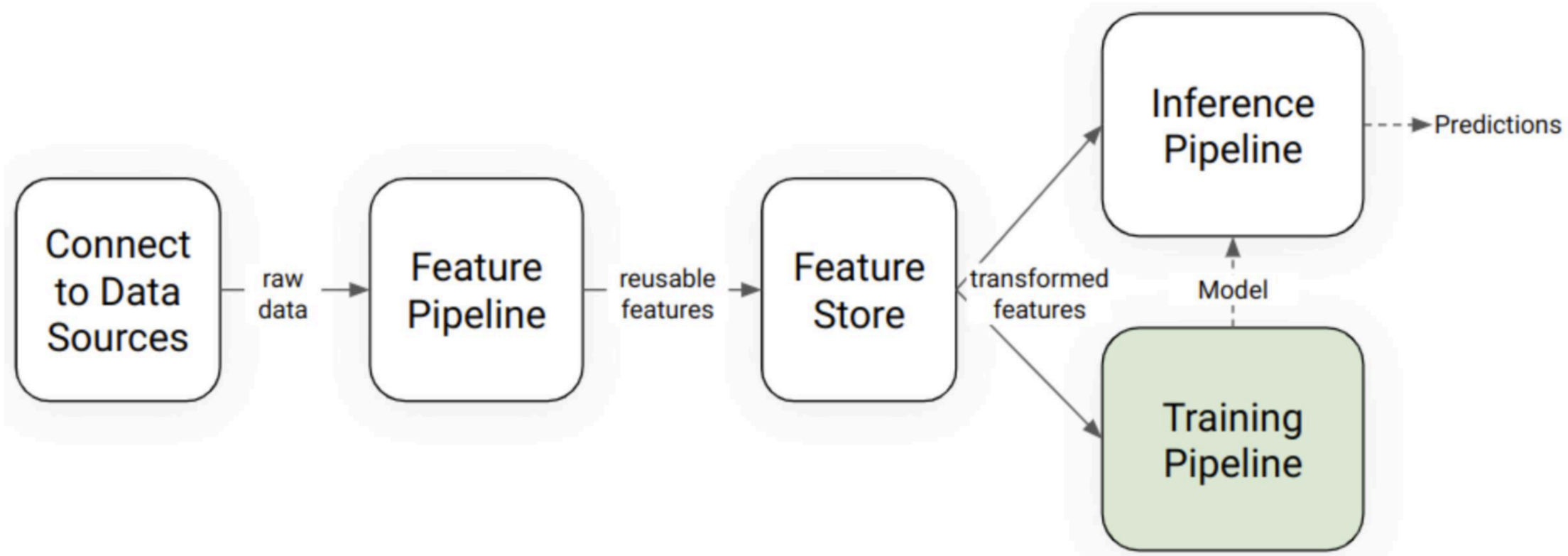
`transactions_4h_aggs` contains aggregated features computed over a 4h time window for each credit card



cc_num	datetime	loc_delta_mavg	trans_volume_mstd	trans_volume_mavg	trans_freq
1111 ...	2022-09-01 11:24	53N,6W	3.4	8	6
1111 ...	2022-09-02 09:17	52N,6W	3.6	3	5
1111 ...	2022-09-04 19:33	51N,7W	4.1	33	45
...	...	...	...	...	...

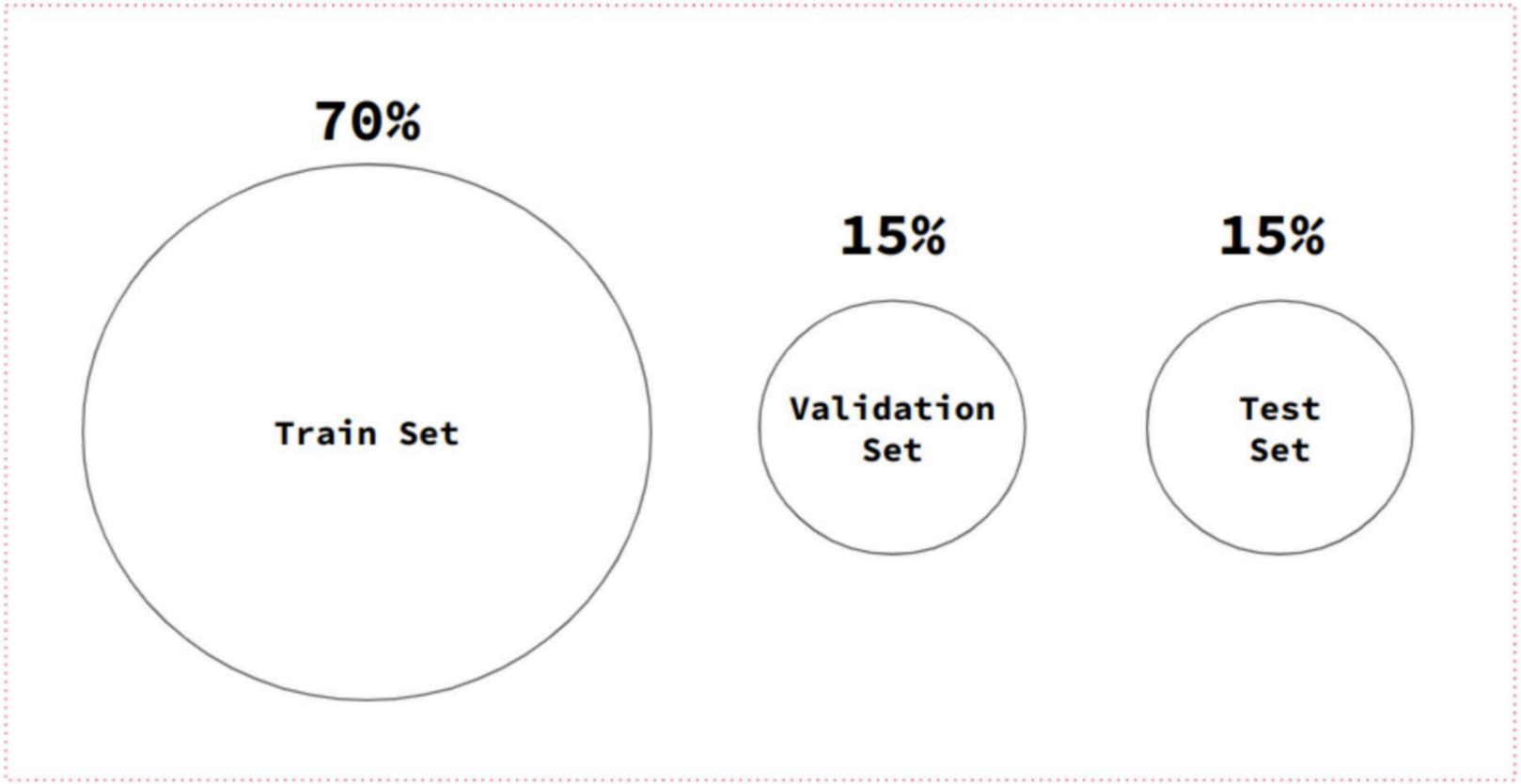
A four-hour window summarizes recent transactions for each credit card

ML Pipeline Stages: Training



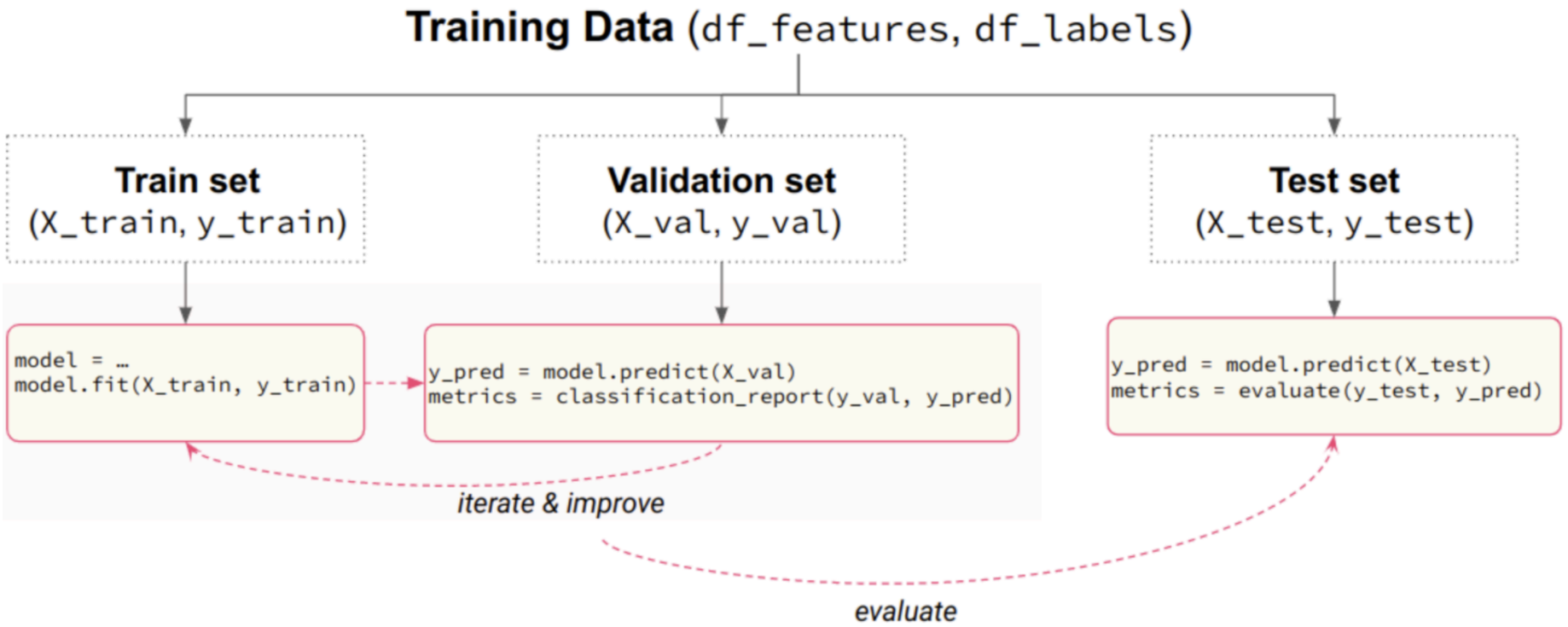
The training pipeline learns a model from stored examples

Training, Validation and Test



Training fits parameters, validation selects candidates and test checks the final choice

Iterative Model Training



The validation loop improves a model while the test set remains separate

Training Code and Evaluation

Original two-way code example; model selection still needs a separate validation step

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import xgboost as xgb

X_train,X_test,y_train,y_test =
    train_test_split(features, labels, test_size=0.2)

model = xgb.XGBClassifier()

model.fit(X_train,y_train)

y_pred = model.predict(X_test)

report_dict = classification_report(
                y_test, y_pred, output_dict=True)
```

Get train and test data sets as features (X) and labels (y)

Use XGBoost as modelling algorithm

Train supervised ML classifier with features and labels from train set

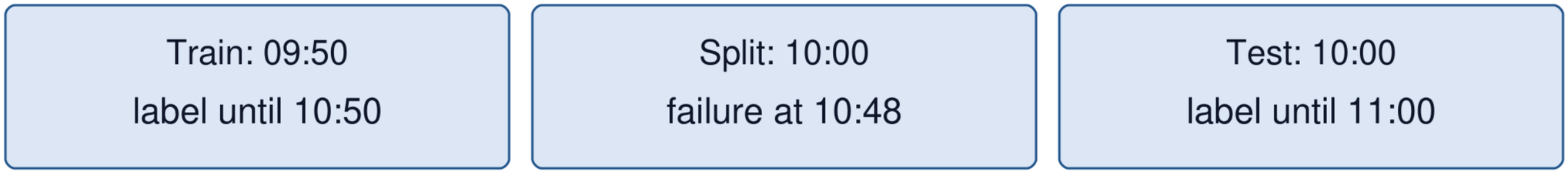
Generate predictions with model on test features (X_test)

Evaluate model performance by comparing predictions (y_pred) and labels (y_test) for the test set

The original code fits a classifier and reports its predictions on held-out data

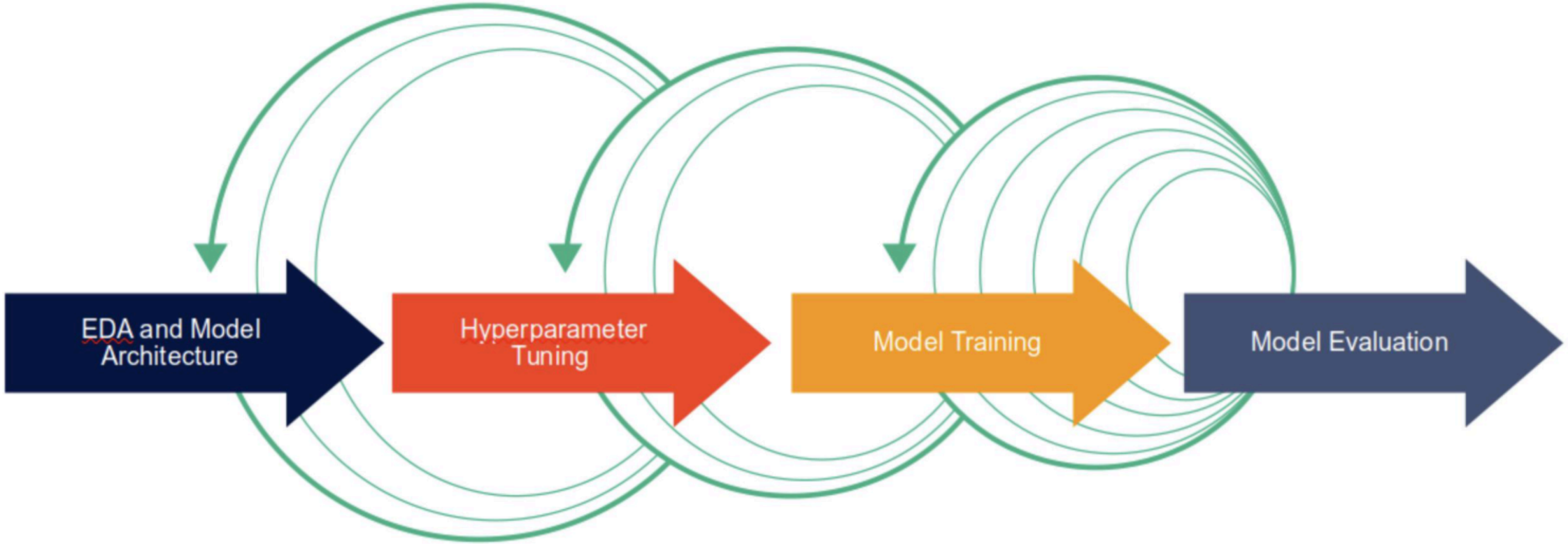
Overlapping Label Windows

Splitting prediction rows can still let the same future failure cross the boundary



The same 10:48 failure labels rows on both sides of the split

Model Development Loops



Architecture, hyperparameters, training and evaluation form nested iteration loops

Accuracy and Missed Failures

A high overall accuracy can hide the failures the system was built to detect



Always normal: 5 misses; detector: 2 misses + 8 false alarms

The 95% model misses every failure in this illustrative set of 100 cases

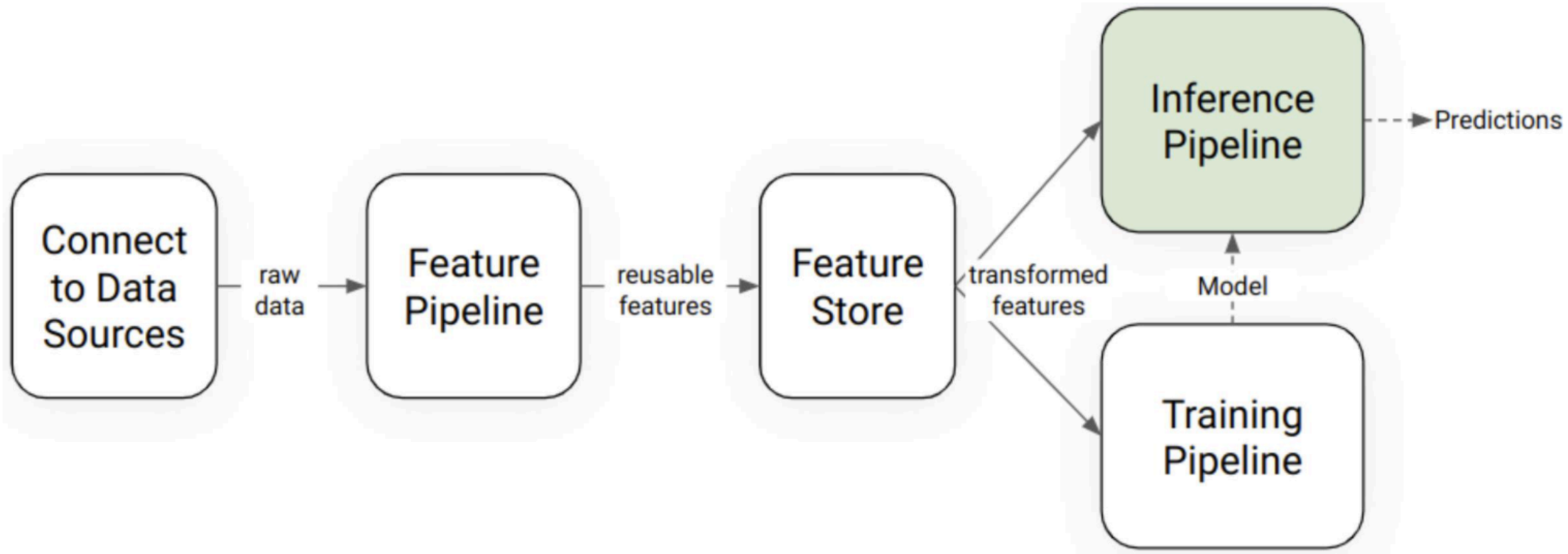
Error Costs and a Decision

Illustrative costs: 100 per missed failure and 1 per false alarm

Policy	Misses	False alarms	Total cost
Always normal	5	0	$5 \times 100 = 500$
Failure detector	2	8	$2 \times 100 + 8 = 208$

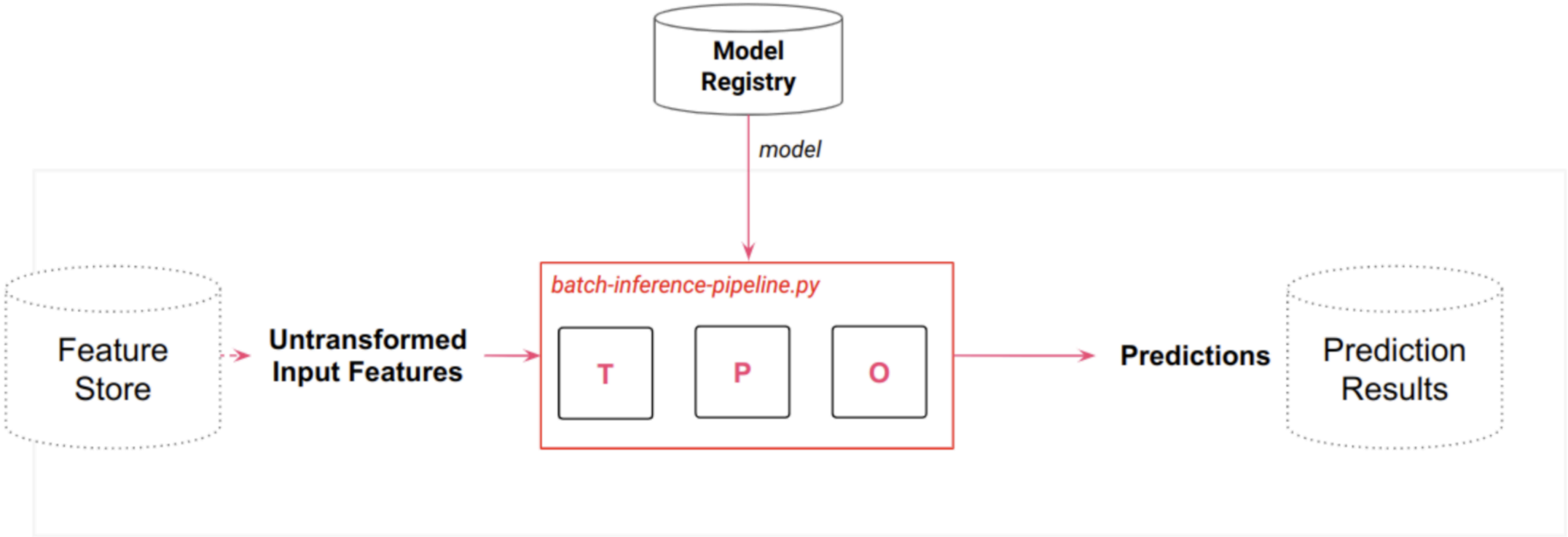
The preferred model changes when the costs of errors are included

ML Pipeline Stages: Inference



The inference pipeline applies the trained model to new inputs

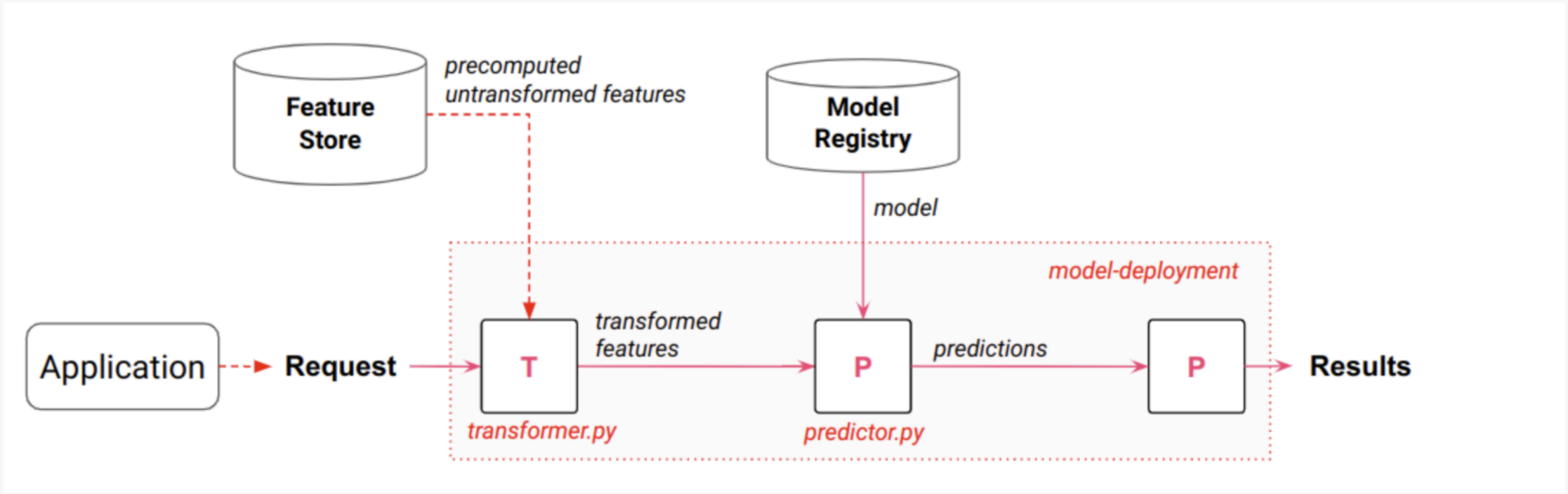
Batch Inference



`TPO = Transform features, Predict, Output.`

Batch inference writes predictions that applications can read later

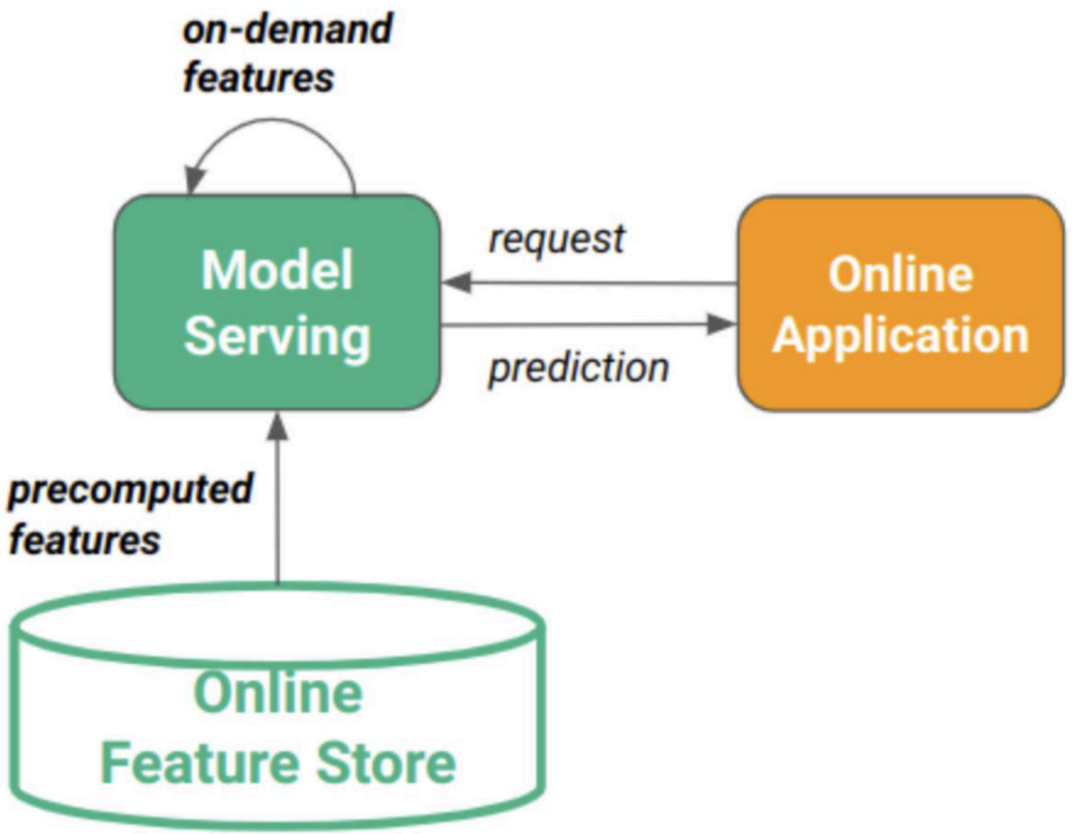
Online Inference



TPP = Transform the input request into features, Predict using input features and the model, Post-process predictions, before output results.

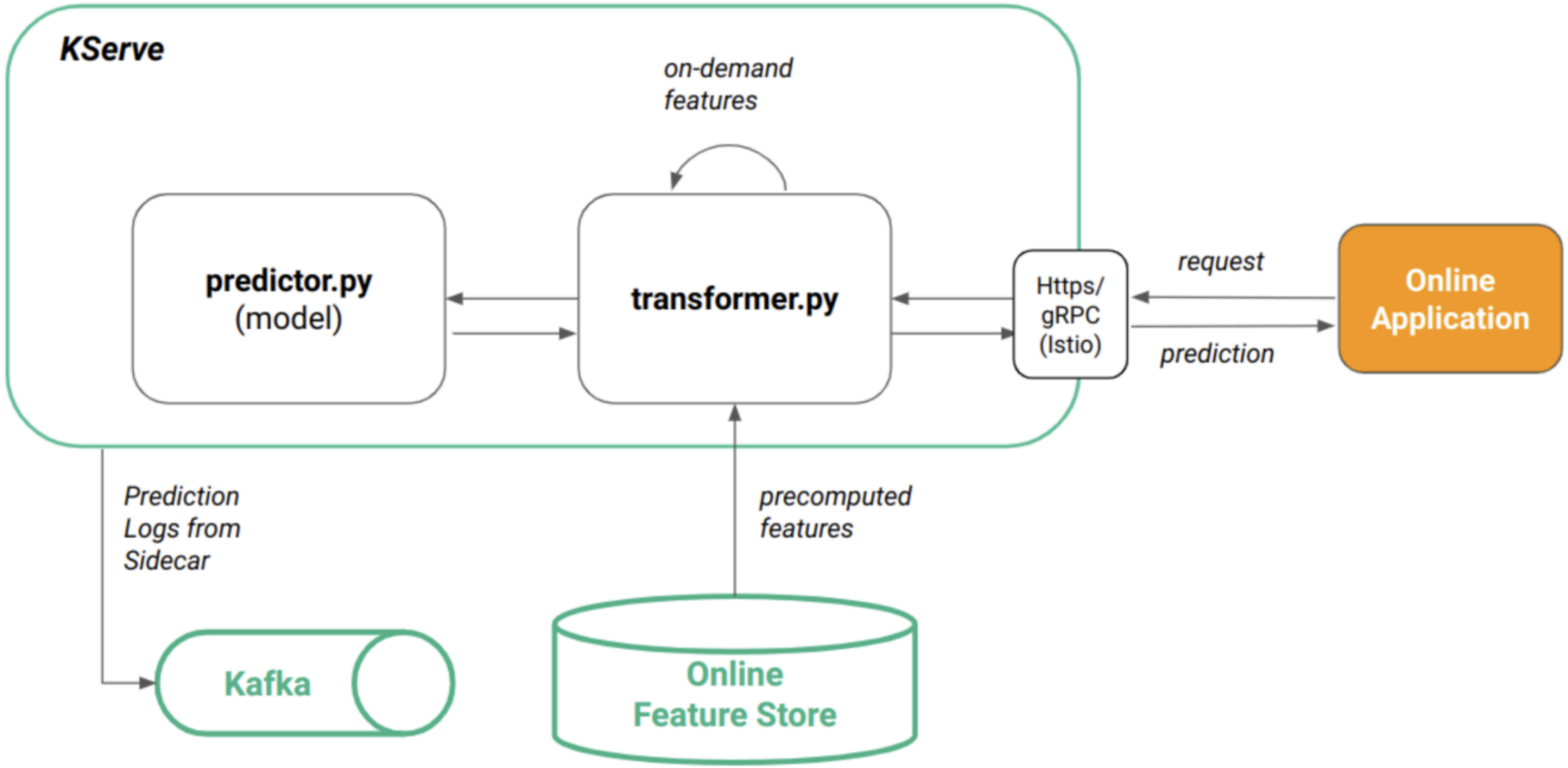
Online inference transforms a request and returns a model prediction

Real-Time Features and Serving



Model serving combines request features with precomputed online features

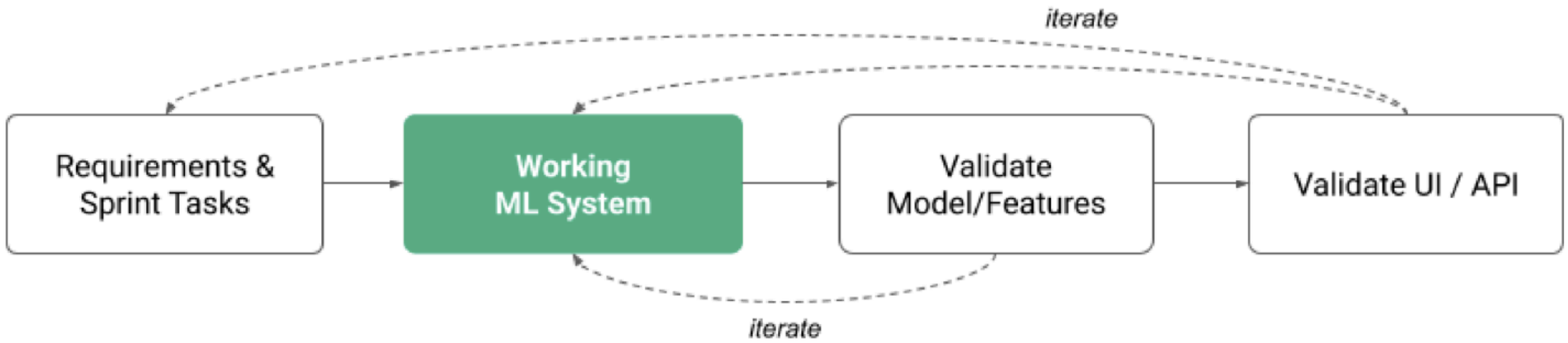
A Real-Time ML Architecture



The serving path connects the predictor, transformer and online feature store

MLOps

- A goal of MLOps is to improve both iteration speed and quality when developing ML



MLOps connects a working ML system to repeated validation and improvement

Why ML Systems Keep Changing

- ML-enabled products **evolve over time**:
  - The available input data (features) change over time
  - The target you are trying to predict changes over time
  - With the help of automation, how can quickly and reliably develop, test, and deploy ML-enabled products without affecting their ongoing operation?
- We should aim to **automate the testing and deployment** of ML-enabled Products

Inputs and targets can change after a model is deployed

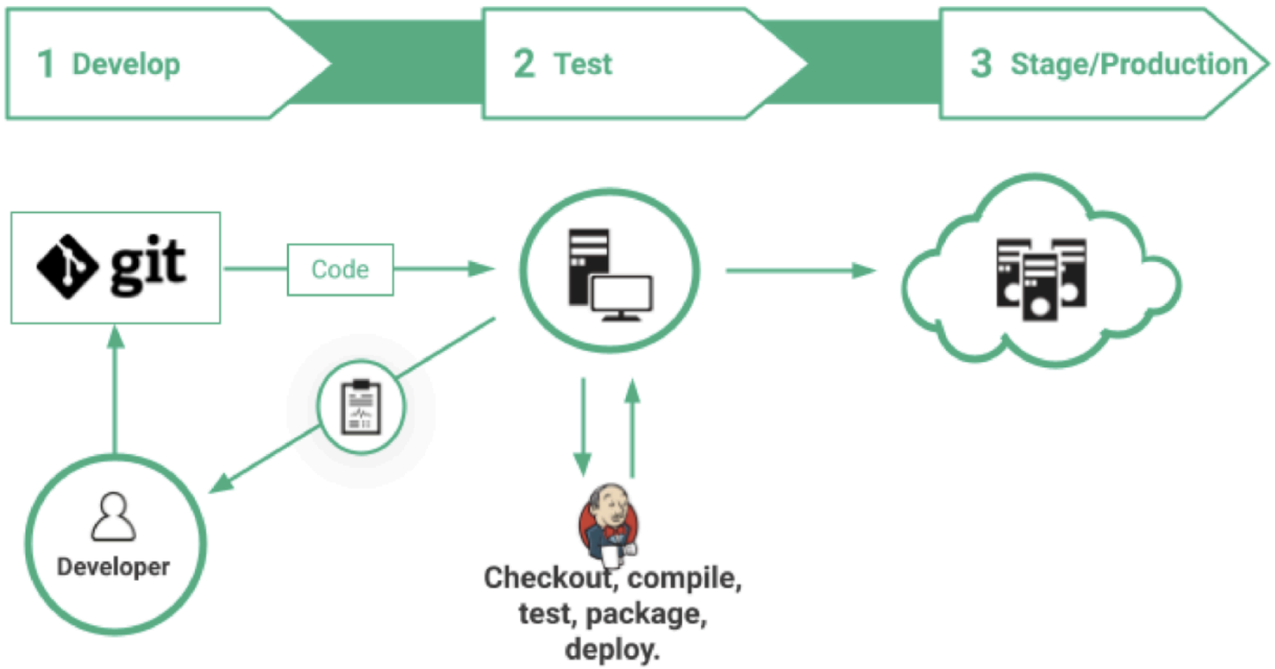
Safe Changes and Shorter Iterations

- Safe incremental updates: make small changes to your source code with confidence that your changes will not break anything (downstream clients, deployments on different platforms), performance regressions, etc)
- Tighter iteration loop: the time taken to run tests or experiments should not dominate the time taken to make the source code changes
- A faster iteration loop makes developers happier and more productive

Small tested changes reduce the risk and cost of repeated development

DevOps

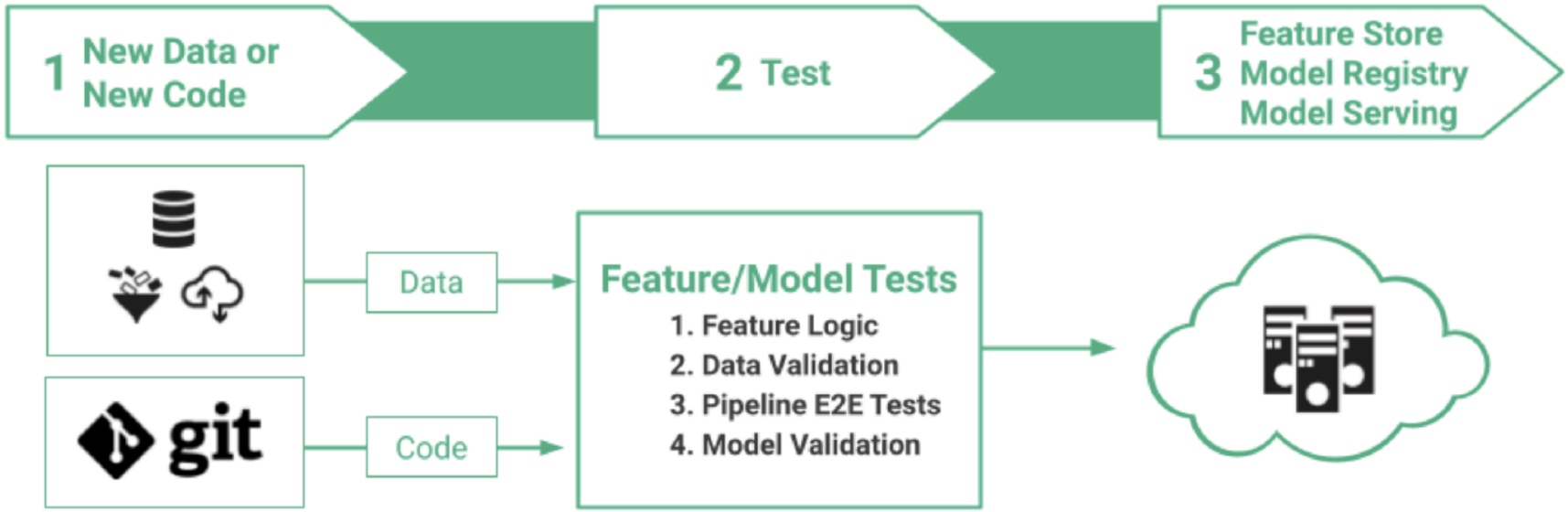
- DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams. Key technologies are **version control, automated testing, versioning** of production deployments.



Development, tests and production delivery are connected through automation

DevOps and MLOps

- In DevOps, changes in source code trigger automated testing and deployment.
- In MLOps, changes in either source code or incoming data trigger automated testing and deployment.



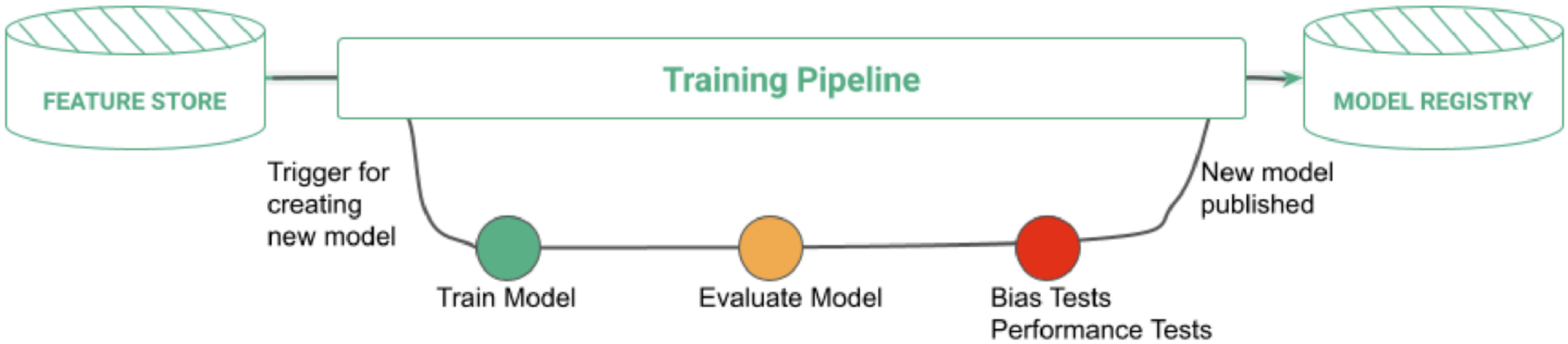
MLOps must check changes in incoming data as well as changes in code

02 · Make Everything Reproducible

01	ML Pipelines and MLOps	from data preparation to deployed predictions
02	**Make Everything Reproducible**	recover data, features, code and models
03	Test Every Step	test features, models and the running system
04	Log Everything	observe predictions and diagnose drift

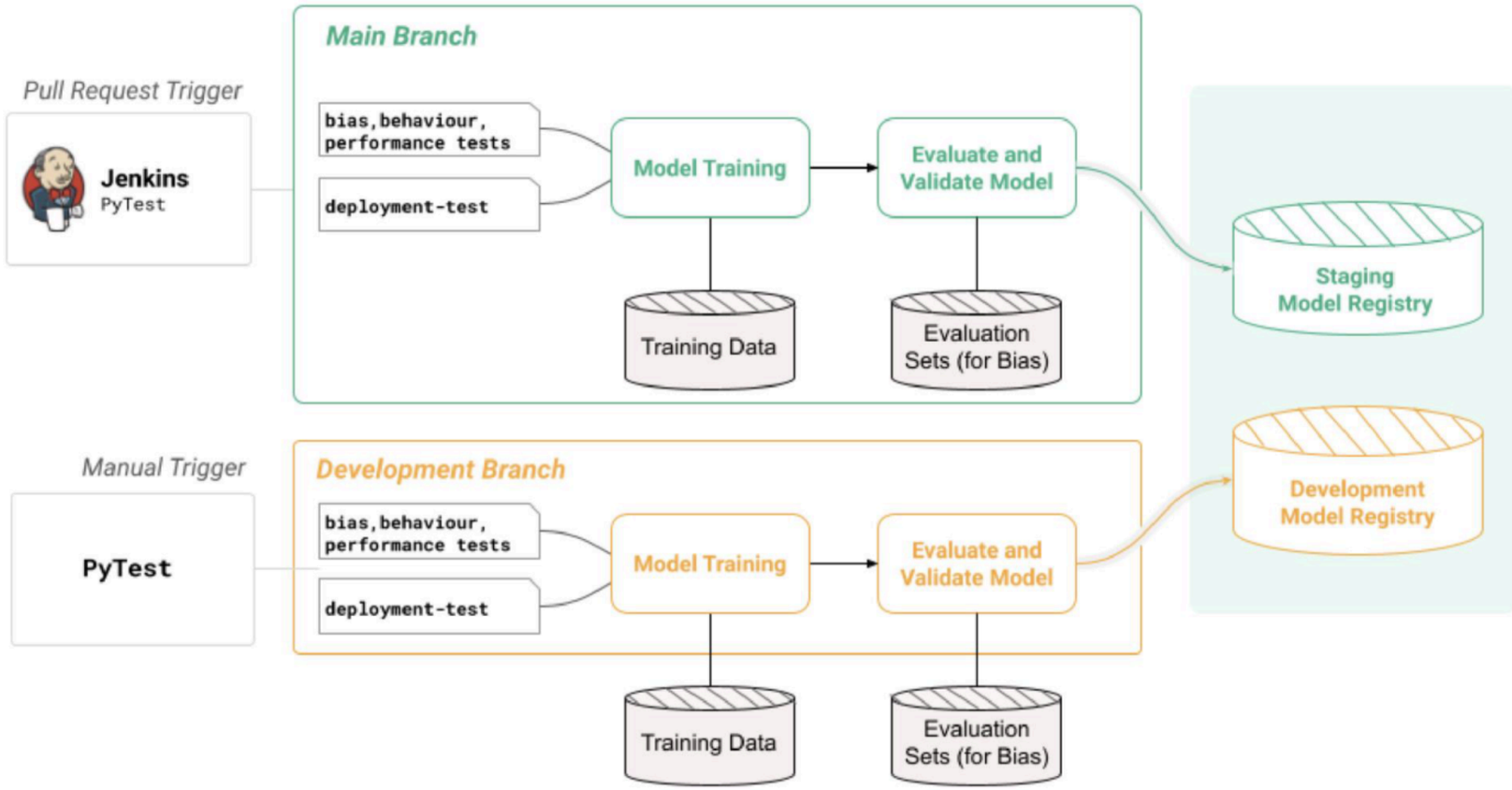
Model Development and Release

- Save the hyperparams and training data used to train the model,
- Only deploy a model that outperforms an existing model deployment (or baseline), assuming both trained on same dataset.
- Only deploy a model if it is free from bias and trained on ethical data.



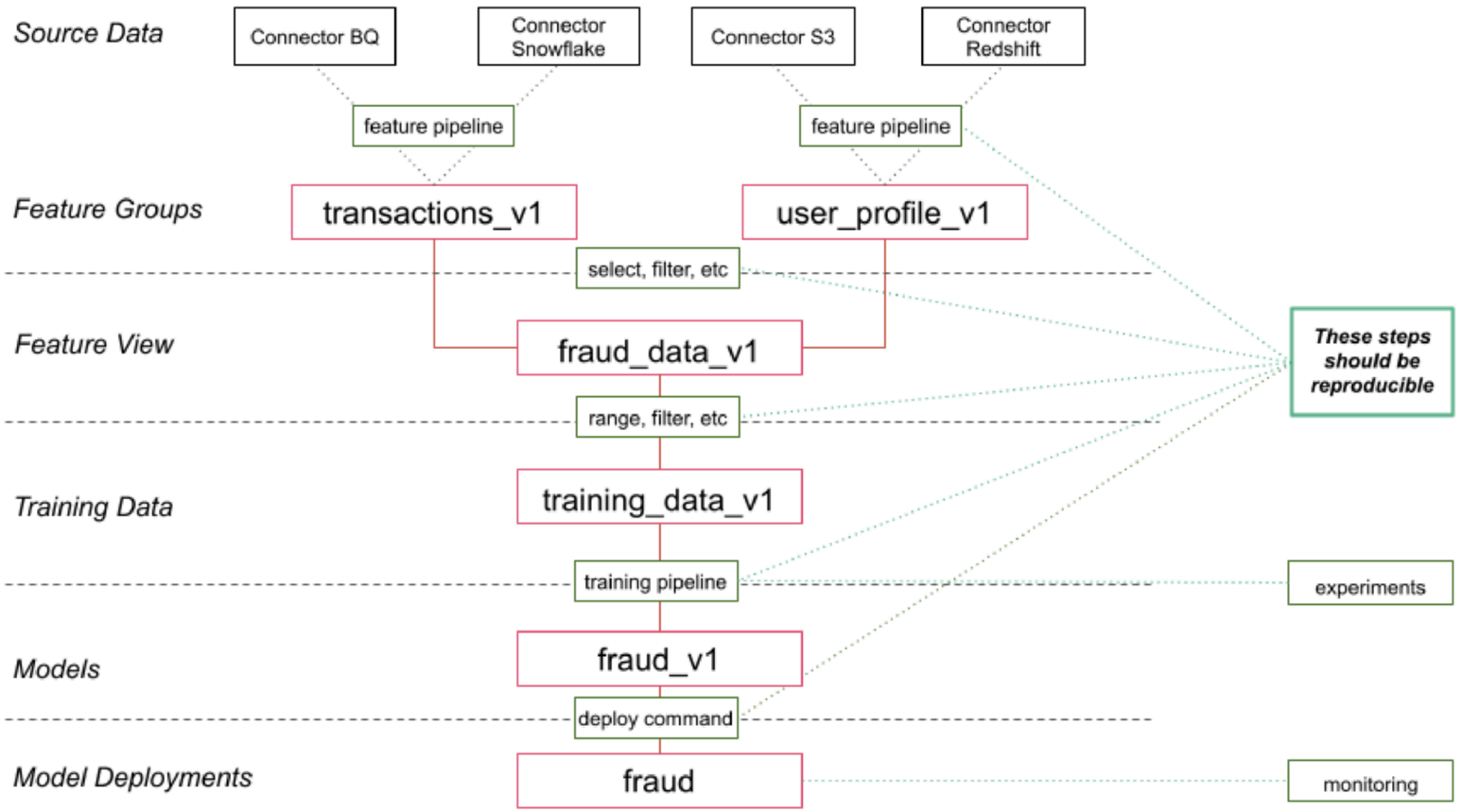
A candidate is compared with the deployed model before release

Development and Main Branches



Development and release paths keep training data and evaluation results together

Reproducible ML



Reproduction requires linked versions of data, feature definitions, code and models

Reproducible ML Assets

ML Asset	Schema Versions	Data Versions	How to reproduce
Feature Groups	Yes	Hudi Commits	Re-run the feature pipeline with the same backfill data
Feature Views	Yes	No	Re-run the same feature view creation commands
Training Data	Yes	No	The parent feature view can recreate training datasets.
Models	Yes	No	Re-run the training pipeline with the same training data,hyperparams, and random number seeds
Model Deployments	No	No	Deploy the same model name/version with the same transformer/predictor code
Prediction Logs	No	No	Not possible if model is stochastic. Emulate by re-running inference pipeline on prediction requests.

Different ML assets need different records to reconstruct an earlier result

The Same Weights, Different Inputs

Changing the feature rule can change the prediction without changing the model

Raw readings	Feature rule	Fixed decision rule
30°C, 32°C	mean 31 / last 32	alarm if x > 31.5

$31 \leq 31.5 \rightarrow$ normal; $32 > 31.5 \rightarrow$ alarm

The mean produces normal; the last reading produces an alarm with unchanged weights

Schema Versioning

```python
from hsfs.feature import Feature

features = [
    Feature(name="id",type="int",online_type="int"),
    Feature(name="name",type="string",online_type="varchar(20)")]

fg = fs.get_feature_group(name="example", version=1)
fg.append_features(features)
```

- Breaking schema changes require updating the schema version for a Feature Group.

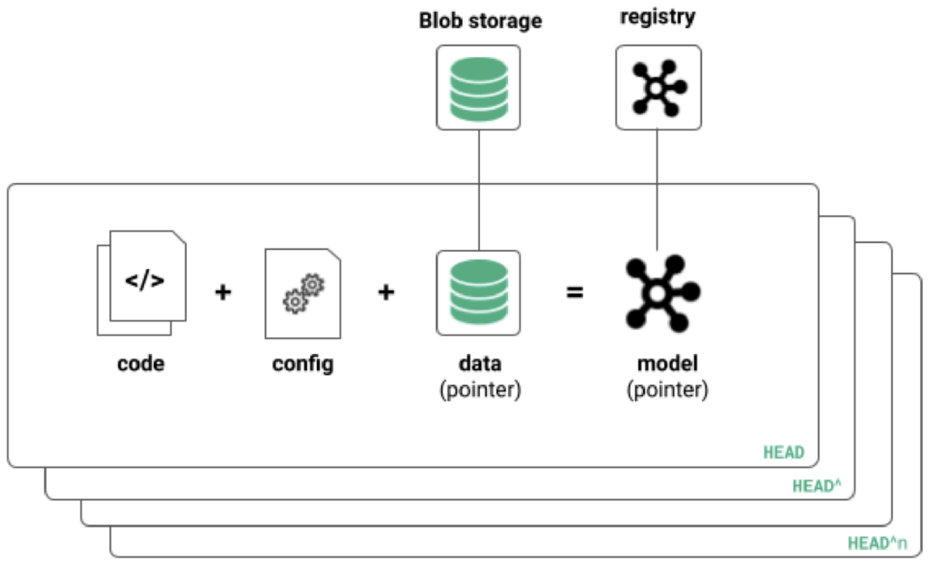
```python
fg1 = fs.create_feature_group(name="example", version=1)
df = fg1.read()
fg2 = fs.create_feature_group(name="example", version=2, features=new_features, …)
fg2.insert(df) #backfill the new feature group with data from the prev version
```

A breaking schema change needs an explicit compatibility and version decision

Data Versioning with Git

2025 tool comparison: scalability depends on the data and storage backend

- Versioning code and data together using Git, such as Data Version Control (DVC).
- Warning: DVC is impractical and inefficient for large amounts of data, as it only stores pointers to blobs (binary large objects) in Git.



Data versions need recoverable content and references alongside the code version

Data Versions with Apache Hudi



View Commits in the Hopsworks UI in the “Activity” tab of a Feature Group

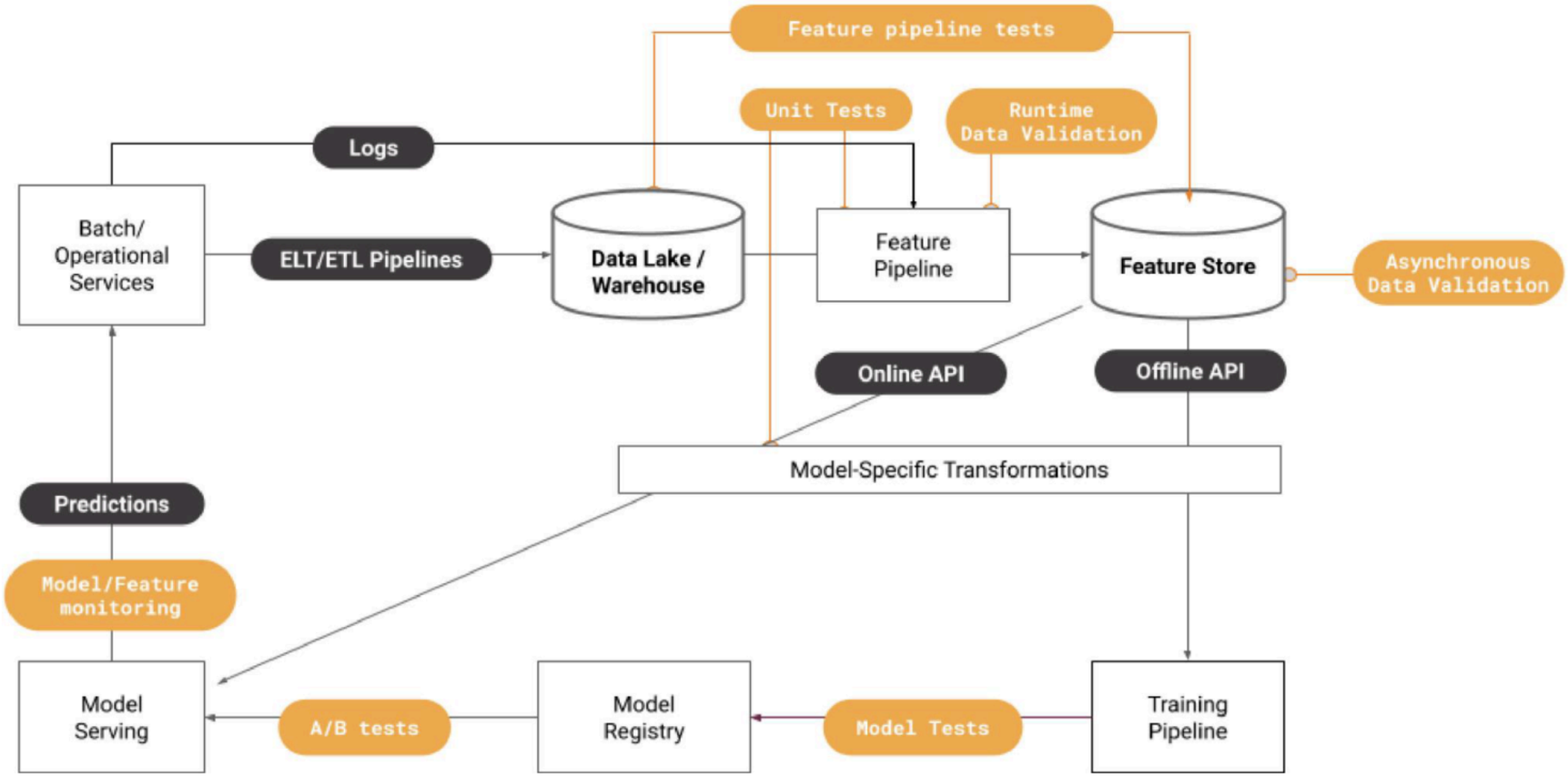


Commit references can identify the data state used by a training pipeline

03 · Test Every Step

01	ML Pipelines and MLOps	from data preparation to deployed predictions
02	Make Everything Reproducible	recover data, features, code and models
03	**Test Every Step**	test features, models and the running system
04	Log Everything	observe predictions and diagnose drift

Tests in ML Systems



Tests cover data, feature code, models, APIs and running predictions

Feature and Model Tests

Feature Tests

- Test that the distributions of each feature match your expectations
- Test the relationship between each feature and the target
- Test the cost of each feature (e.g., latency)
- Test all code that creates input features

Model Tests

- Test that every model specification undergoes a code review
- Test the relationship between offline proxy metrics and the actual impact metrics
- Test the impact of each tunable hyperparameter
- Test the effect of model staleness
- Test against a simpler model as a baseline
- Test model quality and model bias using important data slices (*evaluation sets*)

Feature checks and model checks answer different failure questions

Infrastructure and Monitoring Tests

Infrastructure Tests

- Integration test the feature, training, and inference pipelines.
- Test the reproducibility of model training.
- Test model quality before attempting to serve it.
- Test models via a canary process before production serving.
- Test that a model deployment can be safely rolled back to a previous version.

Monitoring Tests

- Test data quality for features in feature pipelines
- Test that data invariants hold in training and serving inputs
- Test that your training and serving features compute the same values
- Test for model staleness
- Test for NaNs or infinities appearing in your model during training or serving
- Test for regressions in prediction quality on served data

Release tests and production monitoring check behavior beyond offline accuracy

Feature Code before Refactoring

```
def compute_features(df : Dataframe): -> Dataframe
    if config["region"] == "UK":
        df["holidays"] = is_uk_holiday(df["year"], df[" week"])
    else:
        df["holidays"] = is_holiday(df["year"], df["week"])

    df["avg_3wk_spend"] = df["spend"].rolling(3).mean()
    df["acquisition_cost"] = df["spend"] / df["signups"]
    df["spend_shift_3weeks"] = df["spend"].shift(3)
    df["special_feature1"] = compute_bespoke_feature(df)
    df["spend_b"] = multiply_columns(df["acquisition_cost'], df['B'])
    return df

df = loader.load_actuals(dates) # e.g. spend, signups
df = compute_features(df)

feature_group = fs.get_feature_group("customer_features", version=1)
feature_group.insert(df)
```

This feature logic does not contain independently testable features

Data loading, feature logic and writes are mixed in the original function

Testable Feature Functions

```python
def avg_3wk_spend(spend: pd.Series) -> pd.Series:
    """Rolling 3 week average spend."""
    return spend.rolling(3).mean()
……
```

```python
def spend_per_signup(spend: pd.Series, signups: pd.Series) -> pd.Series:
    """The cost per signup in relation to spend."""
    return spend / signups
```

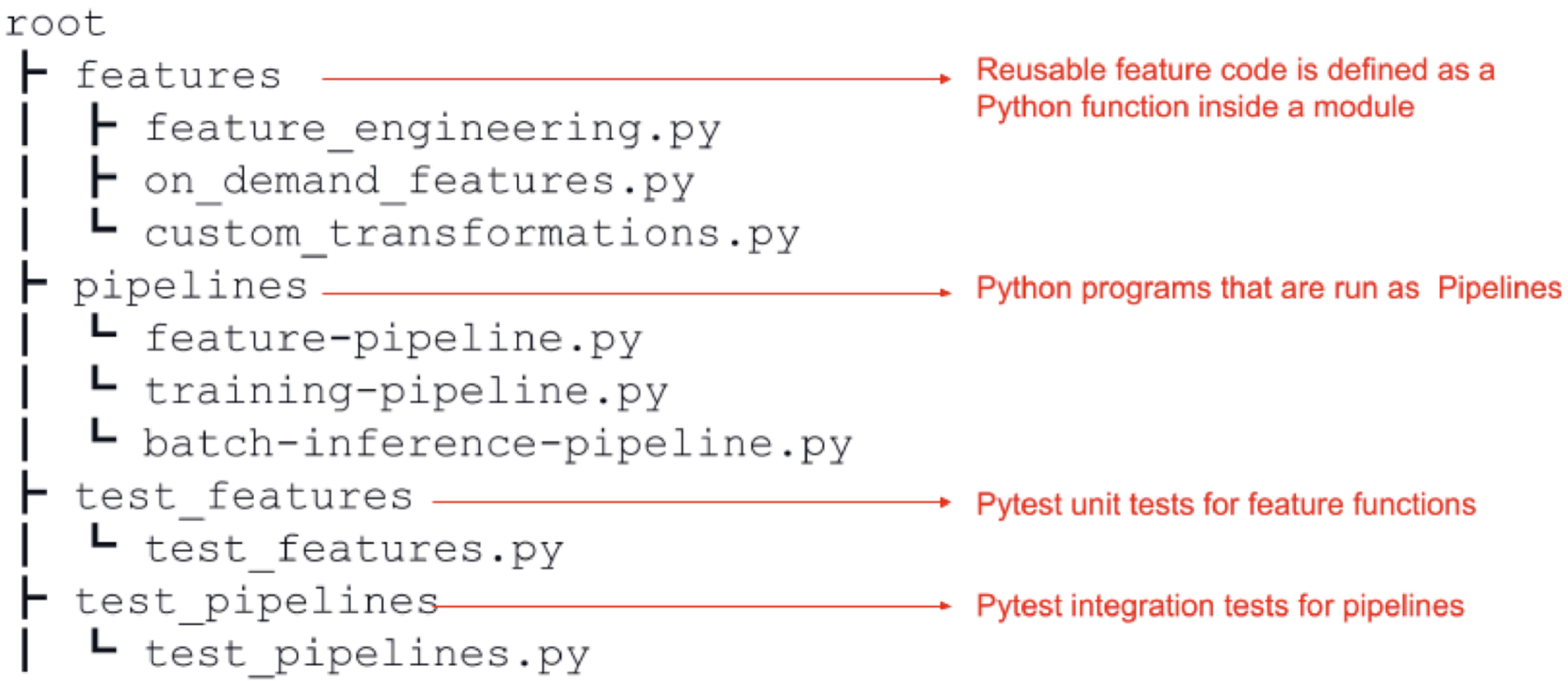
Unit Test

```python
@pytest.fixture
def get_spends(self) -> pd.DataFrame:
    return pd.DataFrame([[20, 40], [5, 4], [4, 10],
                         columns=["spends", "signups", "spend_per_signup"])

def test_spend_per_signup(get_spends : Callable):
    df = get_spends
    df["res"] = spend_per_signup(df["spends"), df("signups")]
    pd.testing.assert_series_equal(df["res"], df["spend_per_signup"])
```

Small pure feature functions make inputs and expected outputs easy to test

A Testable ML Project

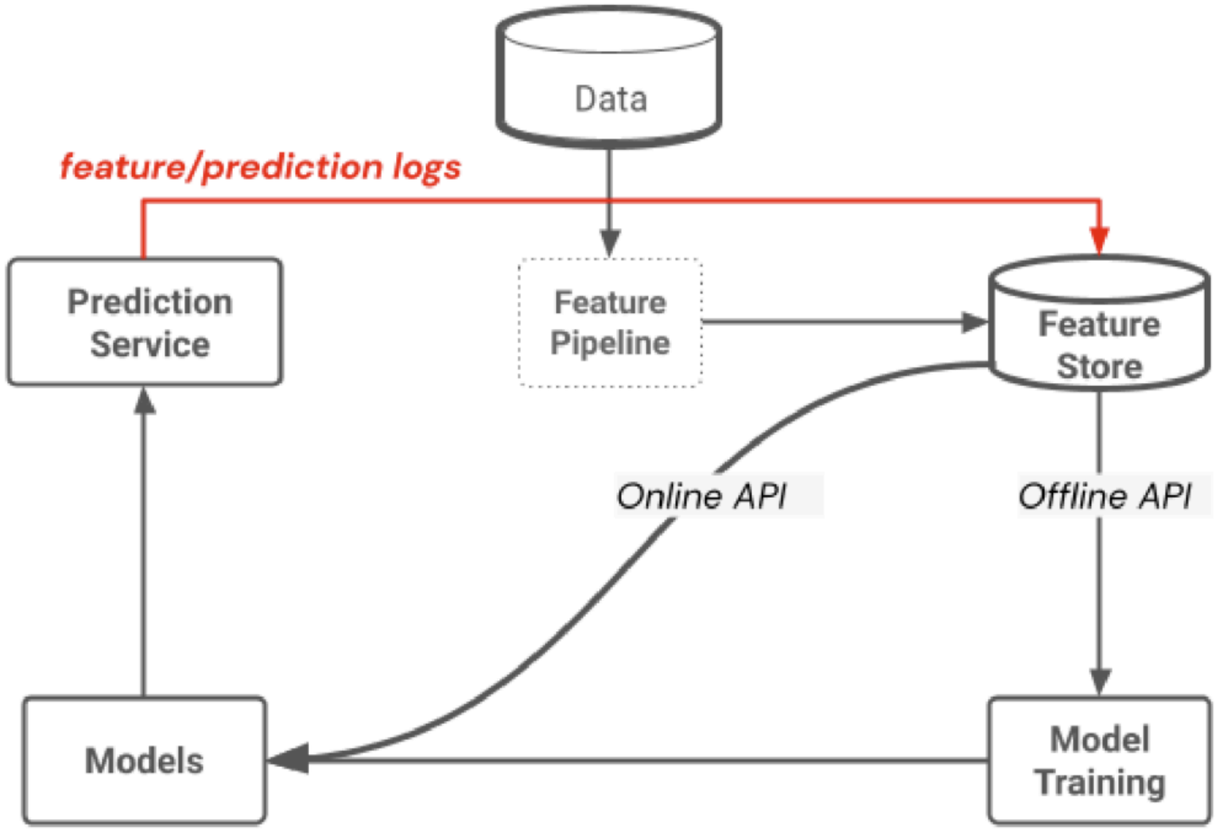


Reusable feature functions, executable pipelines and tests occupy separate modules

04 · Log Everything

01	ML Pipelines and MLOps	from data preparation to deployed predictions
02	Make Everything Reproducible	recover data, features, code and models
03	Test Every Step	test features, models and the running system
04	**Log Everything**	observe predictions and diagnose drift

Feature and Prediction Logs



Feature and prediction logs reconnect source inputs, serving and later training

Uses of Prediction Logs

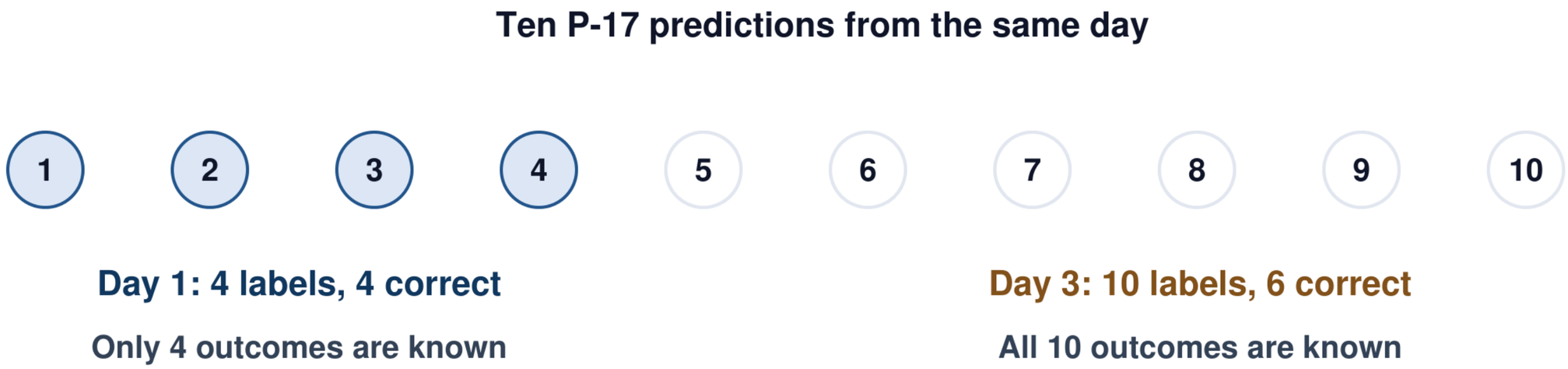
- Logs of predictions and features can be used for:
  - Debugging
  - Model/Feature/Performance Monitoring
  - To create new training data for models
- You should log untransformed feature values and predictions
  - Log before model-specific transformations & reverse_transform predictions



Logs support debugging, monitoring and new training examples

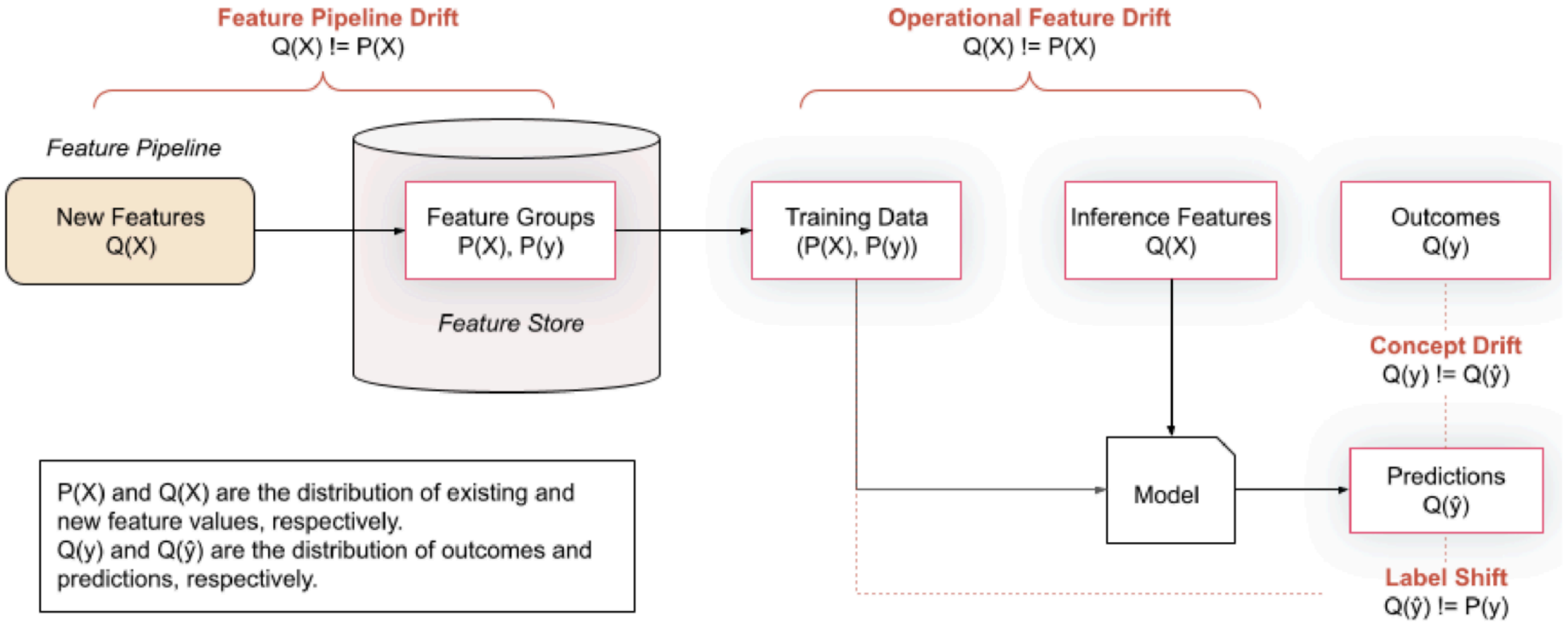
Delayed Outcomes

Recent predictions may still have missing outcomes



The same ten predictions score differently as more labels arrive

Use Case: Data Drift



Pipeline inputs, serving inputs, labels and outcomes can change in different places

Drift Types

Concept drift changes the relationship between inputs and outcomes

- **Feature Pipeline Drift**
  - The distribution of a feature in a DataFrame that is being written to a Feature Group is significantly different from the distribution of that feature stored in the Feature Group.
- **Operational Feature Drift** (aka Covariate Shift)
  - The distribution of a feature in a window of Inference Data is significantly different from the distribution of that feature in the models' training data.
- **Label Shift**
  - The distribution of a window of label data in inference is significantly different from the distribution of that label in the models' training data.
- **Concept Drift**
  - The outcomes are significantly different from the model's predictions.

Different drift signals require different checks before a model update

Takeaway: ML Systems

1. ML pipelines connect data, training and serving; MLOps keeps them working
2. Reproduce results with linked data, feature, code and model versions
3. Test features, models, releases and production behavior
4. Log inputs and predictions to inspect outcomes and drift